

## **SKRIPSI**

# **RANCANG BANGUN SISTEM REKOMENDASI KOS ADAPTIF MENGUNAKAN *HYBRID FILTERING* DAN *USER PROFILING* UNTUK PENANGANAN *COLD-START* PENGGUNA BARU**

Ditulis untuk memenuhi sebagian persyaratan akademik guna memperoleh gelar Sarjana Komputer

Oleh:

**NAMA** : Bryan Daniel Angelino  
**NPM** : 01081230011



**PROGRAM STUDI SISTEM INFORMASI  
FAKULTAS *ARTIFICIAL INTELLIGENCE AND DATA SCIENCE*  
UNIVERSITAS PELITA HARAPAN  
JAKARTA  
2026**

# DAFTAR ISI

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>DAFTAR ISI</b> .....                                 | <b>ii</b> |
| <b>DAFTAR GAMBAR</b> .....                              | <b>iv</b> |
| <b>DAFTAR TABEL</b> .....                               | <b>v</b>  |
| <b>DAFTAR LAMPIRAN</b> .....                            | <b>vi</b> |
| <b>BAB I PENDAHULUAN</b> .....                          | <b>1</b>  |
| 1.1 Latar Belakang .....                                | 1         |
| 1.2 Perumusan Masalah .....                             | 4         |
| 1.3 Pembatasan Masalah .....                            | 4         |
| 1.4 Tujuan dan Manfaat Penelitian .....                 | 5         |
| 1.4.1 Tujuan Penelitian .....                           | 5         |
| 1.4.2 Manfaat Penelitian .....                          | 5         |
| 1.5 Metodologi Penelitian .....                         | 6         |
| 1.5.1 Teknik Pengumpulan Data.....                      | 6         |
| 1.5.2 Metodologi Pengembangan Sistem.....               | 7         |
| 1.6 Sistematika Penulisan .....                         | 9         |
| <b>BAB II LANDASAN TEORI</b> .....                      | <b>11</b> |
| 2.1 Sistem Rekomendasi .....                            | 11        |
| 2.2 Teknik Sistem Rekomendasi.....                      | 11        |
| 2.2.1 <i>Content-Based Filtering</i> .....              | 11        |
| 2.2.2 <i>Collaborative Filtering</i> .....              | 12        |
| 2.2.3 <i>Hybrid Filtering</i> .....                     | 12        |
| 2.2.4 <i>Cosine Similarity</i> .....                    | 14        |
| 2.3 <i>Cold-Start</i> .....                             | 15        |
| 2.4 <i>User Profiling</i> .....                         | 15        |
| 2.5 Metrik Evaluasi .....                               | 17        |
| 2.6 <i>Software Development Life Cycle (SDLC)</i> ..... | 18        |
| 2.7 <i>Prototyping</i> .....                            | 18        |
| 2.8 <i>Unified Modeling Language (UML)</i> .....        | 19        |
| 2.8.1 <i>Use Case Diagram</i> .....                     | 19        |
| 2.8.2 <i>Activity Diagram</i> .....                     | 20        |
| 2.8.3 <i>Class Diagram</i> .....                        | 21        |
| 2.9 Teknologi Pengembangan.....                         | 21        |
| 2.9.1 <i>Flutter</i> dan <i>Dart</i> .....              | 22        |
| 2.9.2 <i>Laravel</i> .....                              | 23        |
| 2.9.3 <i>REST API</i> .....                             | 24        |
| 2.9.4 <i>HTML</i> .....                                 | 25        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 2.9.5 CSS.....                                            | 26        |
| 2.9.6 <i>JavaScript</i> .....                             | 26        |
| 2.9.7 <i>Bootstrap</i> .....                              | 27        |
| 2.9.8 Basis Data .....                                    | 28        |
| 2.9.9 Arsitektur <i>RecommendationService</i> .....       | 29        |
| 2.10 Penelitian Terdahulu .....                           | 29        |
| <b>BAB III SISTEM SAAT INI.....</b>                       | <b>33</b> |
| 3.1 Profil KOSKITA .....                                  | 33        |
| 3.1.1 Visi .....                                          | 33        |
| 3.2 Struktur Organisasi .....                             | 33        |
| 3.3 Analisis Sistem Saat Ini .....                        | 35        |
| 3.3.1 Proses Bisnis Pencarian Kos Saat Ini.....           | 35        |
| 3.3.2 Analisis Aplikasi Sejenis.....                      | 37        |
| 3.4 Identifikasi Permasalahan .....                       | 38        |
| 3.4.1 Kendala Sistem Saat Ini .....                       | 38        |
| <b>BAB IV ANALISIS DAN PERANCANGAN SISTEM USULAN.....</b> | <b>39</b> |
| 4.1 Analisis Kelayakan Sistem.....                        | 39        |
| 4.2 Tahap Analisis.....                                   | 39        |
| 4.2.1 Sistem Usulan (Target Sistem) .....                 | 39        |
| <b>BAB V KESIMPULAN DAN SARAN .....</b>                   | <b>40</b> |
| 5.1 Kesimpulan .....                                      | 40        |
| 5.2 Saran.....                                            | 40        |
| <b>DAFTAR PUSTAKA.....</b>                                | <b>41</b> |

## DAFTAR GAMBAR

|                                                                        |    |
|------------------------------------------------------------------------|----|
| Gambar 1. 1 Kerangka Penelitian.....                                   | 8  |
| Gambar 3. 1 Struktur Organisasi KOSKITA .....                          | 34 |
| Gambar 3. 2 <i>Activity Diagram</i> Proses Pencarian Kos Saat Ini..... | 36 |

## DAFTAR TABEL

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Tabel 2. 2 Contoh Perhitungan Skor <i>Hybrid</i> .....            | 13 |
| Tabel 2. 3 Notasi <i>Use Case Diagram</i> .....                   | 20 |
| Tabel 2. 4 Notasi <i>Activity Diagram</i> .....                   | 21 |
| Tabel 2. 5 Notasi <i>Class Diagram</i> .....                      | 21 |
| Tabel 2. 6 Rangkuman Penelitian Terdahulu .....                   | 31 |
| Tabel 3. 1 Perbandingan Fitur Rekomendasi pada Aplikasi Kos ..... | 37 |

## **DAFTAR LAMPIRAN**

|                                              |   |
|----------------------------------------------|---|
| LAMPIRAN A HASIL PERCAKAPAN SEWA KOS.....    | 1 |
| LAMPIRAN B LEMBAR MONITORING BIMBINGAN ..... | 1 |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Pertumbuhan jumlah mahasiswa dan pekerja yang merantau ke kota-kota pusat pendidikan dan industri meningkatkan kebutuhan akan hunian sewa sementara berupa kamar kos. Calon penyewa dihadapkan pada ribuan pilihan kos yang beragam dari sisi lokasi, harga, fasilitas, tipe kos (putra/putri/campur), serta aturan hunian, sehingga proses memilih kos yang paling sesuai menjadi tidak mudah. Banyaknya pilihan ini menimbulkan masalah kelebihan informasi (*information overload*) yang membuat pengambilan keputusan menjadi melelahkan dan rentan kurang optimal [1]. Situasi ini semakin menyulitkan karena calon penyewa harus membandingkan puluhan hingga ratusan *listing* kos secara manual tanpa panduan yang disesuaikan dengan kebutuhan pribadi mereka. Akibatnya, proses pencarian kos yang seharusnya singkat dapat berlarut-larut hingga memerlukan survei lokasi maupun komunikasi berulang dengan pemilik kos sebelum akhirnya ditemukan hunian yang benar-benar sesuai.

Aplikasi pencarian kos seperti Mamikos, Rukita, dan Papikost telah membantu pengguna menelusuri ketersediaan kos melalui fitur pencarian dan penyaringan (*filter*), namun fitur tersebut umumnya masih bersifat manual, dimana pengguna harus menentukan kriteria sendiri dan membandingkan banyak *listing* satu per satu, sementara hasil yang ditampilkan belum tentu personal terhadap kebutuhan masing-masing individu. Sistem rekomendasi (*recommender system*) hadir sebagai solusi untuk menyaring dan menyajikan opsi yang paling relevan secara otomatis bagi tiap pengguna [2], [3]. Pendekatan berbasis *filter* menempatkan seluruh beban penyaringan pada pengguna, sehingga dua pengguna dengan kebutuhan yang berbeda tetap menerima daftar hasil yang identik selama menggunakan kriteria pencarian yang sama. Kondisi ini semakin bermasalah bagi pengguna baru yang belum memahami wilayah atau belum memiliki gambaran jelas mengenai preferensinya sendiri, sehingga proses coba-coba menjadi sulit dihindari. Kondisi ini pada akhirnya mendorong kebutuhan akan sistem yang mampu mempelajari preferensi pengguna secara otomatis, alih-alih hanya mengandalkan kriteria yang diinput secara manual.

Ketidakpuasan pengguna terhadap hasil rekomendasi yang kurang personal bukan sekadar dugaan, melainkan gejala yang telah terdokumentasi pada berbagai platform. Literatur sistem rekomendasi mencatat bahwa kepuasan pengguna cenderung menurun

ketika konten yang direkomendasikan gagal selaras dengan preferensi personalisasi mereka, karena banyak algoritma cenderung mengulang item yang sudah pernah dilihat atau diklik sehingga mengurangi kebaruan dan nilai yang dirasakan dari rekomendasi tersebut [4]. Gejala serupa turut teramati pada aplikasi pencarian kos di Indonesia. Analisis sentimen terhadap ulasan pengguna aplikasi Mamikos mengidentifikasi tiga belas permasalahan pengguna yang tersebar pada aspek harga, proses, dan produk layanan [5]. Temuan ini menunjukkan bahwa sekadar menyediakan fitur pencarian dan penyaringan belum cukup untuk menjawab kebutuhan personalisasi pengguna, sehingga pendekatan yang lebih adaptif terhadap karakteristik masing-masing pengguna menjadi relevan untuk dikaji lebih lanjut. Persoalan ini menjadi dasar bagi penelitian ini untuk mengkaji bagaimana pendekatan yang lebih adaptif terhadap karakteristik individu dapat diterapkan pada domain pencarian kos, khususnya di Indonesia yang memiliki karakteristik hunian dan preferensi penyewa yang khas.

Dua teknik inti pada sistem rekomendasi adalah *content-based filtering*, yang merekomendasikan item berdasarkan kemiripan atributnya dengan preferensi pengguna, dan *collaborative filtering* (CF), yang merekomendasikan berdasarkan pola perilaku pengguna lain yang serupa. Pendekatan *hybrid* menggabungkan keduanya agar saling menutupi kelemahan masing-masing dan terbukti meningkatkan akurasi rekomendasi [6], [7], [8]. Di Indonesia, penerapan CF dan *hybrid filtering* telah banyak diteliti pada berbagai domain [9], [10], meskipun demikian, CF memiliki kelemahan berupa masalah *cold-start*, yaitu ketika sistem belum dapat memberikan rekomendasi yang baik bagi pengguna atau item baru yang belum memiliki riwayat interaksi [11]. Masalah ini menjadi krusial pada aplikasi yang baru menambahkan fitur rekomendasi, karena sebagian besar pengguna awal secara alami tergolong pengguna baru yang belum memiliki jejak interaksi sama sekali.. Kelemahan ini menjadi pertimbangan utama dalam merancang mekanisme yang tetap dapat berfungsi baik pada kondisi data interaksi minim maupun melimpah

Pada domain kos di Indonesia, sistem rekomendasi yang ada umumnya berfokus pada atribut item atau mengandalkan riwayat pengguna lain [12], [13], dan belum menyesuaikan rekomendasi dengan profil penyewa sekaligus menangani kondisi pengguna baru, padahal pemanfaatan profil dan karakteristik pengguna terbukti meningkatkan relevansi rekomendasi, dan penggabungan data profil, demografis, serta lokasi efektif untuk mengatasi *cold-start* [14]. Karakteristik penyewa kos sangat beragam, mulai dari lajang hingga berkeluarga dan dari mahasiswa hingga pekerja, sementara kos di Indonesia memiliki atribut khas seperti tipe putra/putri/campur serta aturan jam malam,



sehingga profil penyewa (*user profiling*) menjadi acuan yang baik untuk memberikan rekomendasi yang adaptif, khususnya bagi pengguna baru yang belum memiliki riwayat interaksi. Tanpa mekanisme yang memanfaatkan profil sejak awal, pengguna baru berisiko menerima rekomendasi yang seragam dan kurang relevan, sehingga berpotensi menurunkan minat mereka untuk terus menggunakan aplikasi pada tahap paling krusial, yaitu pengalaman pertama. Oleh karena itu, penelitian ini berupaya mengisi kesenjangan tersebut dengan merancang mekanisme yang memanfaatkan profil pengguna sejak proses pendaftaran. Rancangan ini diharapkan dapat menjawab kebutuhan spesifik pengguna sejak awal, tanpa harus menunggu terkumpulnya riwayat interaksi yang memadai.

Kombinasi *hybrid filtering* dan *user profiling* seperti ini sebenarnya bukan hal baru secara umum, namun penerapannya sejauh ini masih terkonsentrasi pada domain seperti film [14], *e-commerce*, atau *point of interest*. Pada domain kos di Indonesia, penelitian rekomendasi yang ada masih mengandalkan teknik tunggal seperti *collaborative filtering* [13] atau klusterisasi [10], sementara kombinasi *hybrid filtering* dengan *user profiling* untuk domain ini belum ditemukan dalam literatur yang ditelaah. Kesenjangan ini penting untuk diisi karena karakteristik pencarian kos berbeda dari domain lain: keputusan penyewa dipengaruhi kombinasi faktor tipe kos, aturan kos, fasilitas, lokasi, dan kondisi personal yang jarang muncul bersamaan pada domain film atau *e-commerce*. Studi terkini menunjukkan bahwa pendekatan *hybrid filtering* yang digabungkan dengan *user profiling* tetap menjadi fondasi yang solid untuk sistem rekomendasi domain-spesifik seperti kos, sambil membuka peluang peningkatan lebih lanjut pada penelitian mendatang, sehingga diperlukan pendekatan yang menyesuaikan rekomendasi dengan karakteristik pengguna sejak awal penggunaan aplikasi, bukan hanya setelah pengguna memiliki riwayat interaksi yang cukup. Pendekatan semacam ini sejalan dengan kebutuhan aplikasi pencarian kos yang menargetkan mahasiswa dan pekerja baru di suatu kota, yang secara alami merupakan pengguna dengan riwayat interaksi minim namun kebutuhan pencarian yang mendesak.

Berdasarkan latar belakang tersebut, penelitian ini merancang dan membangun modul sistem rekomendasi kos adaptif berbasis *hybrid filtering* yang memanfaatkan *user profiling* untuk menangani kondisi *cold-start* pada pengguna baru sebagai bagian dari pengembangan aplikasi KOSKITA yang telah beroperasi. Kebaruan penelitian ini terletak pada penggabungan *hybrid filtering* dan *user profiling* secara khusus pada domain pencarian kos di Indonesia, kombinasi yang sejauh penelusuran literatur belum banyak diterapkan pada domain ini. Penelitian ini tidak hanya berfokus pada pengembangan model rekomendasi, tetapi juga mengimplementasikannya ke dalam sebuah prototipe

modul sehingga hasil rekomendasi dapat ditampilkan secara langsung kepada pengguna KOSKITA. Dengan pendekatan ini, penelitian diharapkan mampu memberikan rekomendasi yang lebih relevan bagi pengguna baru dibandingkan pendekatan yang hanya mengandalkan riwayat interaksi, sekaligus menjadi referensi bagi pengembangan sistem rekomendasi pada domain pencarian kos di Indonesia. Dengan demikian, kontribusi penelitian ini bersifat ganda, yaitu memperkaya kajian akademik mengenai kombinasi hybrid filtering dan user profiling pada domain yang belum banyak diteliti, sekaligus memberikan manfaat praktis bagi pengembangan aplikasi KOSKITA secara langsung.

## 1.2 Perumusan Masalah

Berdasarkan latar belakang, perumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana merancang sistem rekomendasi kos berbasis *hybrid filtering* yang menggabungkan atribut kos, profil penyewa, dan data interaksi pengguna?
2. Bagaimana strategi *cold-start* berbasis profil penyewa dapat digunakan untuk memberikan rekomendasi kepada pengguna baru yang belum memiliki riwayat interaksi?
3. Bagaimana performa model *hybrid* yang diusulkan dibandingkan *baseline* popularitas, *content-based filtering*, dan *collaborative filtering* pada skenario *warm-start* dan *cold-start*?
4. Bagaimana hasil rekomendasi ditampilkan dalam prototipe antarmuka KOSKITA agar dapat dipahami oleh pengguna?

## 1.3 Pembatasan Masalah

Agar penelitian terarah, ditetapkan batasan masalah sebagai berikut.

1. Penelitian hanya mengimplementasikan modul mesin rekomendasi kos dan prototipe antarmukanya; fitur pembayaran, *chat*, pembukuan, dan manajemen properti berada di luar ruang lingkup.
2. Studi kasus dibatasi pada data kos di wilayah Tangerang dan sekitarnya (area kampus).
3. Data profil pengguna diperoleh langsung dari proses registrasi dan pengisian profil oleh responden pada prototipe aplikasi KOSKITA, bukan melalui survei terpisah. Sebanyak 150 responden yang merupakan calon penyewa kos menggunakan modul rekomendasi secara langsung; hasil rekomendasi yang diterima kemudian dinilai

oleh responden, dan penilaian tersebut menjadi dasar ground truth bagi evaluasi model.

4. Evaluasi model dilakukan menggunakan metrik *Precision@K*, *Recall@K*, *NDCG*, dan *MAP* berdasarkan *ground truth* hasil penilaian responden, dengan item yang mendapat penilaian di atas ambang tertentu dikategorikan sebagai item relevan. Usabilitas prototipe antarmuka diukur menggunakan *System Usability Scale* (SUS).
5. Atribut yang digunakan meliputi harga, lokasi/jarak, tipe kos, fasilitas, dan peraturan kos, serta profil penyewa (mahasiswa/pekerja, gender, anggaran, dan status tinggal).

#### **1.4 Tujuan dan Manfaat Penelitian**

Bagian ini menguraikan tujuan yang ingin dicapai melalui penelitian ini beserta manfaat yang diharapkan, baik secara teoretis maupun praktis.

##### **1.4.1 Tujuan Penelitian**

Tujuan penelitian ini dirumuskan sebagai berikut.

1. Merancang model rekomendasi kos berbasis *hybrid filtering* yang menggabungkan atribut kos, profil penyewa, dan data interaksi pengguna.
2. Merancang dan menguji strategi penanganan *cold-start* berbasis profil penyewa bagi pengguna baru.
3. Mengukur dan membandingkan performa model *hybrid* terhadap metode *baseline*.
4. Membangun prototipe antarmuka KOSKITA untuk menampilkan hasil rekomendasi agar dapat dipahami pengguna.

##### **1.4.2 Manfaat Penelitian**

Secara teoretis, penelitian ini memberikan kontribusi pada penerapan *hybrid filtering* dan *user profiling* untuk penanganan *cold-start* pada domain kos di Indonesia yang masih jarang diteliti. Secara praktis, hasil penelitian dapat menjadi dasar pengembangan fitur rekomendasi pada aplikasi pencarian kos sehingga membantu calon penyewa, khususnya pengguna baru, menemukan kos yang sesuai dengan lebih cepat dan relevan. Kontribusi teoretis ini diharapkan dapat menjadi rujukan bagi penelitian selanjutnya yang ingin mengeksplorasi kombinasi teknik serupa pada domain hunian sewa lain di luar kos, seperti apartemen atau rumah kontrakan, yang memiliki karakteristik pencarian yang sebanding. Manfaat praktis ini secara langsung mendukung tujuan pengembangan KOSKITA sebagai platform yang tidak hanya menyediakan listing kos, tetapi juga membantu pengguna mengambil keputusan secara lebih efisien.

Kontribusi ini melengkapi literatur yang selama ini lebih banyak berfokus pada domain film, e-commerce, dan pariwisata dibandingkan domain hunian sewa.

Selain memberikan manfaat bagi calon penyewa, hasil penelitian ini juga dapat menjadi referensi bagi pengembang aplikasi pencarian kos dalam mengembangkan fitur rekomendasi yang lebih personal. Bagi pemilik kos, sistem rekomendasi yang lebih relevan juga berpotensi membantu mempertemukan properti dengan calon penyewa yang memiliki kebutuhan yang sesuai. Dengan mempertemukan kebutuhan kedua belah pihak secara lebih tepat sasaran, sistem rekomendasi ini berpotensi mengurangi tingkat kekosongan kos yang berkepanjangan sekaligus mempercepat proses transaksi penyewaan pada platform KOSKITA. Manfaat ini menunjukkan bahwa dampak penelitian tidak hanya dirasakan oleh satu pihak, melainkan turut memberi nilai tambah bagi ekosistem KOSKITA secara keseluruhan. Nilai tambah ini pada akhirnya berkontribusi pada keberlanjutan platform KOSKITA sebagai penghubung antara calon penyewa dan pemilik kos.

## **1.5 Metodologi Penelitian**

Bagian ini menjelaskan metodologi yang digunakan dalam penelitian ini, mencakup teknik pengumpulan data serta metodologi pengembangan sistem yang diterapkan.

### **1.5.1 Teknik Pengumpulan Data**

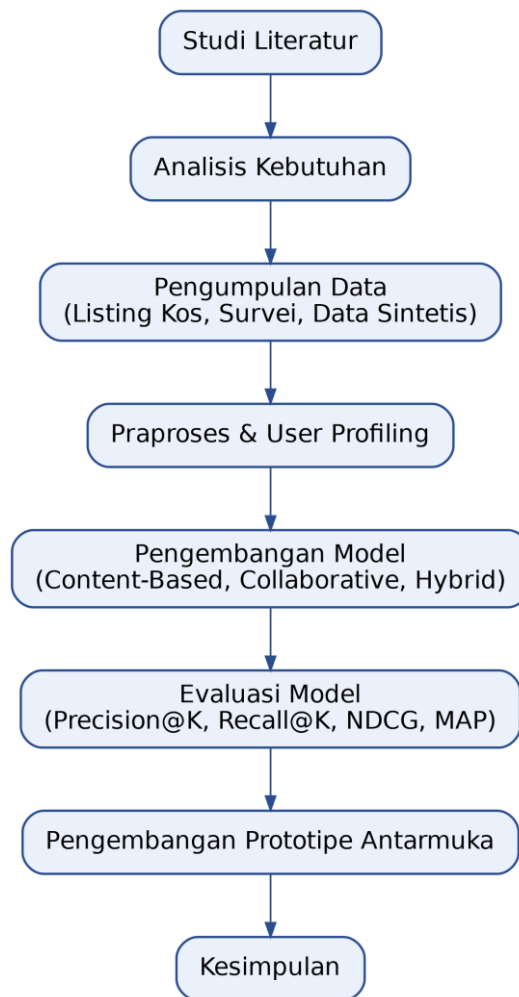
Pengumpulan data pada penelitian ini dilakukan melalui studi literatur serta interaksi langsung 150 responden, yaitu calon penyewa kos, dengan prototipe modul rekomendasi pada aplikasi KOSKITA. Berbeda dari pendekatan survei terpisah, data profil pengguna diperoleh langsung dari proses registrasi dan pengisian profil pengguna yang dilakukan responden pada aplikasi, mencakup status sebagai mahasiswa atau pekerja, jenis kelamin, kisaran anggaran, serta preferensi fasilitas yang diinginkan. Data profil ini langsung digunakan sebagai masukan bagi proses *user profiling* tanpa perantara survei atau formulir terpisah. Pendekatan ini dipilih agar data yang digunakan untuk membangun profil benar-benar mencerminkan perilaku pengguna sesungguhnya pada aplikasi, bukan sekadar jawaban hipotetis terhadap skenario survei yang mungkin berbeda dari perilaku penggunaan yang sebenarnya. Dengan demikian, kualitas data yang menjadi masukan *model* rekomendasi dapat lebih terjamin sejak tahap pengumpulan awal.

Setelah profil terbentuk, sistem menghasilkan rekomendasi kos menggunakan *content-based filtering*, *collaborative filtering*, dan penggabungan *hybrid* yang dihitung di backend Laravel. Responden kemudian memberikan penilaian terhadap kos yang

direkomendasikan; penilaian inilah yang menjadi *ground truth* bagi perhitungan *Precision@K*, *Recall@K*, *NDCG*, dan *MAP*, dengan kos yang mendapat penilaian di atas ambang tertentu dikategorikan sebagai item relevan. Setelah menyelesaikan pengujian, responden mengisi kuesioner *System Usability Scale* (SUS) untuk mengukur usability antarmuka, serta pertanyaan tambahan mengenai kepuasan terhadap kualitas rekomendasi yang diterima. Rangkaian tahapan ini memastikan bahwa evaluasi tidak hanya mengukur akurasi algoritmik semata, tetapi juga mempertimbangkan pengalaman pengguna secara langsung terhadap prototipe yang dibangun. Kombinasi kedua jenis evaluasi ini memberikan gambaran yang lebih lengkap mengenai keberhasilan sistem, baik dari sisi teknis maupun dari sisi penerimaan pengguna.

### 1.5.2 Metodologi Pengembangan Sistem

Pengembangan sistem mengikuti *model Prototyping* dalam kerangka *Software Development Life Cycle* (SDLC), yang memungkinkan penyempurnaan rancangan secara iteratif berdasarkan umpan balik pada tiap tahapan pengembangan. Tahapan mencakup analisis kebutuhan, perancangan *model* rekomendasi *hybrid* beserta mekanisme *cold-start*, implementasi prototipe, serta pengujian dan evaluasi performa *model* bersama 150 responden melalui skema pengujian pengguna (*user testing*). Setiap tahapan dilakukan secara berulang berdasarkan umpan balik yang diperoleh, sehingga rancangan *model* rekomendasi maupun antarmuka aplikasi dapat terus disempurnakan sebelum mencapai bentuk akhir prototipe yang diujikan kepada responden. Sifat iteratif ini memungkinkan setiap kekurangan yang ditemukan pada satu tahap segera diperbaiki sebelum berlanjut ke tahap berikutnya. Pendekatan ini dipilih karena sesuai dengan karakteristik penelitian yang menghasilkan produk berupa prototipe, bukan sistem final yang langsung siap produksi.



Gambar 1. 1 Kerangka Penelitian.

Kerangka penelitian pada Gambar 1. 1 menggambarkan tahapan penelitian dari studi literatur hingga penarikan kesimpulan. Penelitian diawali dengan studi literatur untuk memahami teori sistem rekomendasi, teknik *hybrid filtering*, dan strategi penanganan *cold-start*, dilanjutkan dengan analisis kebutuhan untuk merumuskan fitur yang harus dipenuhi sistem. Tahapan berikutnya mencakup perancangan *model* rekomendasi dan pengumpulan data, sebelum masuk ke tahap implementasi, pengujian, serta penarikan kesimpulan berdasarkan hasil evaluasi yang diperoleh. Urutan tahapan ini disusun agar setiap keputusan desain pada tahap awal memiliki dasar yang kuat sebelum diteruskan ke tahap implementasi yang lebih teknis.

Tahap pengumpulan data menghimpun data *listing* kos serta data profil dan penilaian rekomendasi dari 150 responden yang diperoleh langsung melalui registrasi dan penggunaan prototipe aplikasi. Data tersebut kemudian melalui praproses dan pembentukan profil pengguna (*user profiling*) sebelum digunakan pada tahap pengembangan *model*, yang mencakup komponen *content-based filtering*, *collaborative filtering*, dan penggabungan keduanya melalui strategi *hybrid*. Setiap komponen model dikembangkan secara bertahap dan diuji kinerjanya sebelum diintegrasikan menjadi satu

kesatuan mekanisme hybrid yang siap dievaluasi. Pendekatan bertahap ini memudahkan identifikasi sumber permasalahan apabila performa salah satu komponen tidak sesuai harapan pada tahap pengujian akhir. Data ini menjadi dasar bagi seluruh tahap pemodelan dan evaluasi yang dilakukan pada penelitian selanjutnya.

*Model* yang dihasilkan dievaluasi menggunakan metrik *Precision@K*, *Recall@K*, *NDCG*, dan *MAP* pada skenario *warm-start* maupun *cold-start* berdasarkan *ground truth* hasil penilaian 150 responden, dilengkapi pengukuran usabilitas antarmuka menggunakan *System Usability Scale* (SUS), sebelum diintegrasikan ke dalam prototipe antarmuka KOSKITA. Kerangka ini memastikan setiap tahap penelitian saling terhubung secara sistematis, dari landasan teori hingga produk akhir berupa prototipe yang dapat diuji. Dengan keterhubungan ini, setiap keputusan desain pada tahap awal penelitian dapat ditelusuri dampaknya hingga ke hasil akhir prototipe yang diuji bersama responden. Tahap evaluasi ini menjadi penentu apakah pendekatan *hybrid* yang diusulkan benar-benar memberikan peningkatan performa dibandingkan metode *baseline* yang lebih sederhana. Hasil evaluasi ini pada akhirnya menjadi dasar bagi kesimpulan mengenai efektivitas pendekatan yang diusulkan pada penelitian ini.

## **1.6 Sistematika Penulisan**

Struktur penulisan dalam tugas akhir ini disusun secara sistematis dengan pembagian sebagai berikut.

### **BAB I : PENDAHULUAN**

Bab ini menjelaskan latar belakang penelitian, perumusan masalah, pembatasan masalah, tujuan dan manfaat penelitian, metodologi penelitian, serta sistematika penulisan yang digunakan dalam penyusunan tugas akhir ini.

### **BAB II : LANDASAN TEORI**

Bab ini menguraikan teori-teori yang mendasari penelitian, meliputi sistem rekomendasi, teknik *content-based filtering* dan *collaborative filtering*, *hybrid filtering*, pengukuran kemiripan (*cosine similarity*), *cold-start* dan *user profiling*, metrik evaluasi, metodologi pengembangan perangkat lunak, *Unified Modeling Language* (UML), serta teknologi pengembangan yang digunakan. Bab ini juga memuat rangkuman penelitian terdahulu yang menjadi rujukan penelitian ini.

### **BAB III : SISTEM SAAT INI**

Bab ini membahas profil KOSKITA, struktur organisasi, serta analisis terhadap sistem pencarian kos yang berjalan saat ini, termasuk identifikasi permasalahan yang menjadi dasar perancangan sistem usulan.

#### **BAB IV : ANALISIS DAN PERANCANGAN SISTEM USULAN**

Bab ini menguraikan analisis kelayakan sistem serta perancangan sistem rekomendasi kos yang diusulkan, mencakup perancangan *model hybrid filtering* dan prototipe antarmuka aplikasi.

#### **BAB V : KESIMPULAN DAN SARAN**

Bab ini memuat kesimpulan yang diperoleh dari keseluruhan proses penelitian serta saran bagi pengembangan penelitian maupun sistem selanjutnya.



## BAB II

### LANDASAN TEORI

#### 2.1 Sistem Rekomendasi

Sistem rekomendasi adalah perangkat lunak yang menyaring informasi untuk memprediksi preferensi pengguna terhadap suatu item dan menyajikan item yang paling relevan. Secara umum, pendekatannya dikelompokkan menjadi *content-based filtering*, *collaborative filtering*, dan *hybrid* [15], [2]. Sistem rekomendasi banyak diterapkan pada *e-commerce*, hiburan, dan layanan berbasis lokasi untuk mengatasi kelebihan informasi dan meningkatkan pengalaman pengguna [18]. Pada domain hunian sewa seperti kos, penerapan sistem rekomendasi masih tergolong baru dibandingkan dengan domain lain, sehingga kajian mengenai teknik yang paling sesuai untuk karakteristik domain ini menjadi relevan untuk dieksplorasi lebih lanjut. Pada domain hunian sewa seperti kos, penerapan sistem rekomendasi masih tergolong baru dibandingkan domain lain, sehingga kajian mengenai teknik yang paling sesuai untuk karakteristik domain ini menjadi relevan untuk dieksplorasi lebih lanjut.

#### 2.2 Teknik Sistem Rekomendasi

Bagian ini menguraikan teknik-teknik utama yang mendasari sistem rekomendasi, yaitu *content-based filtering*, *collaborative filtering*, *hybrid filtering*, serta pengukuran kemiripan yang menjadi dasar perhitungan skor rekomendasi pada penelitian ini.

##### 2.2.1 Content-Based Filtering

*Content-based filtering* merekomendasikan item dengan mencocokkan atribut item terhadap profil preferensi pengguna. Pada *Content-Based Filtering*, preferensi pengguna dan atribut kos direpresentasikan sebagai vektor fitur berdasarkan karakteristik seperti kisaran harga, lokasi, dan fasilitas. Tingkat kemiripan antara profil pengguna dan setiap kos kemudian dihitung menggunakan metode *cosine similarity* untuk menghasilkan nilai kesesuaian yang menjadi dasar rekomendasi [15]. Kelebihannya adalah tidak bergantung pada data pengguna lain sehingga relatif tahan terhadap *cold-start item*, namun cenderung menghasilkan rekomendasi yang kurang beragam [7]. Pada penelitian ini, *content-based filtering* berperan penting khususnya bagi pengguna baru, karena rekomendasi tetap dapat dihasilkan hanya berdasarkan atribut kos dan profil pengguna tanpa memerlukan riwayat interaksi sama sekali. Peran ini menjadikan *content-based filtering* sebagai komponen utama pada tahap awal penggunaan aplikasi, sebelum data interaksi mulai terkumpul.

### 2.2.2 Collaborative Filtering

*Collaborative filtering* (CF) merekomendasikan item berdasarkan pola kesamaan antarpengguna atau antaritem. Penelitian ini menggunakan pendekatan CF berbasis memori, yang menghitung kemiripan secara langsung dari matriks interaksi (mis. *user-based* atau *item-based*) menggunakan ukuran seperti *cosine similarity* atau korelasi Pearson [15], [7]. Pendekatan ini dipilih karena dapat dihitung langsung tanpa proses pelatihan *model* yang kompleks, sehingga sesuai dengan skala data prototipe dan dapat diimplementasikan langsung pada aplikasi *mobile* tanpa pustaka pembelajaran mesin tambahan. Pemanfaatan karakteristik pengguna pada CF berbasis profil juga terbukti meningkatkan relevansi rekomendasi [19]. Namun demikian, ketergantungan CF pada data interaksi menjadikannya kurang optimal bagi pengguna atau item yang baru bergabung, sehingga diperlukan mekanisme pelengkap agar sistem tetap dapat memberikan rekomendasi yang bermakna sejak awal.

### 2.2.3 Hybrid Filtering

Pada penelitian ini pendekatan *hybrid* dipilih karena mampu memanfaatkan keunggulan *content-based filtering* dalam memahami karakteristik kos dan *collaborative filtering* dalam memanfaatkan pola interaksi pengguna. Kombinasi keduanya diharapkan dapat menghasilkan rekomendasi yang lebih stabil dibandingkan apabila hanya menggunakan satu metode saja. Secara spesifik, penelitian ini menerapkan *switching hybrid* dengan komponen *weighted* di dalamnya: strategi *switching* menentukan komponen mana yang aktif berdasarkan ketersediaan data (sepenuhnya *content-based* saat *cold-start*, atau kombinasi keduanya saat data interaksi tersedia), sedangkan strategi *weighted* mengatur proporsi kontribusi *content-based* dan *collaborative filtering* pada kondisi kedua tersebut. Secara matematis, komponen *weighted* dirumuskan sebagai berikut. Pemilihan strategi *switching-weighted* ini didasarkan pada kebutuhan untuk tetap dapat menghasilkan rekomendasi meskipun data interaksi belum tersedia, sekaligus memanfaatkan data tersebut secara optimal ketika sudah ada.

$$\text{Skor\_hybrid} = \alpha \times \text{Skor\_CB} + (1 - \alpha) \times \text{Skor\_CF}$$

Formula tersebut mengindikasikan bahwa Skor\_CB adalah skor dari komponen *content-based filtering*, Skor\_CF adalah skor dari komponen *collaborative filtering*, dan  $\alpha$  ( $0 \leq \alpha \leq 1$ ) adalah bobot yang mengatur kontribusi masing-masing komponen pada kondisi *warm-start*. Pada kondisi *cold-start*, mekanisme *switching* menetapkan  $\alpha = 1$  secara otomatis, sehingga skor sepenuhnya berasal dari *content-based filtering* tanpa perlu menghitung Skor\_CF yang datanya belum tersedia. Nilai  $\alpha$  dapat ditentukan melalui

eksperimen untuk memperoleh kombinasi yang menghasilkan performa terbaik pada data pengujian. Nilai  $\alpha$  dapat ditentukan melalui eksperimen untuk memperoleh kombinasi yang menghasilkan performa terbaik pada data pengujian.

Formula dibawah menjelaskan bahwa  $A \cdot B$  adalah hasil kali titik (*dot product*) antara vektor A dan B, sedangkan  $\|A\|$  dan  $\|B\|$  adalah norma (panjang) masing-masing vektor. Nilai *cosine similarity* berkisar antara 0 hingga 1 untuk vektor bernilai non-negatif, dengan nilai 1 menandakan kedua vektor identik arah dan 0 menandakan tidak ada kemiripan. Pada penelitian ini, *cosine similarity* digunakan pada dua konteks: pertama, untuk mengukur kemiripan antara vektor profil/preferensi penyewa dengan vektor atribut kos pada komponen *content-based filtering*; kedua, untuk mengukur kemiripan antarpengguna atau antarkos pada komponen *collaborative filtering* berbasis memori. Pemilihan *cosine similarity* dibandingkan ukuran jarak lain seperti Euclidean distance didasarkan pada karakteristiknya yang tidak terpengaruh oleh skala atau panjang vektor, sehingga lebih sesuai untuk data atribut kos dan profil pengguna yang memiliki rentang nilai bervariasi. Karakteristik ini menjadikan *cosine similarity* sebagai pilihan yang konsisten untuk diterapkan pada kedua komponen *model hybrid* dalam penelitian ini.

$$\cos(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$$

*Cosine similarity* mengukur kemiripan antara dua vektor berdasarkan sudut di antara keduanya, bukan besarnya vektor tersebut. Ukuran ini banyak digunakan pada sistem rekomendasi karena mampu menangani vektor berdimensi tinggi dan jarang, seperti representasi atribut kos maupun profil pengguna [15], [19]. Karakteristik ini menjadikan *cosine similarity* lebih stabil dibandingkan ukuran kemiripan berbasis jarak ketika diterapkan pada data dengan banyak nilai kosong, seperti riwayat interaksi pengguna yang baru menggunakan sebagian kecil dari seluruh kos yang tersedia. Kemiripan cosine antara dua vektor A dan B dirumuskan sebagai berikut.

Sebagai ilustrasi, Tabel 2. 2 menunjukkan contoh perhitungan skor *hybrid* untuk satu kos pada dua kondisi berbeda. Contoh perhitungan ini bertujuan memperjelas bagaimana persamaan (2.1) diterapkan secara konkret pada kasus nyata.

Tabel 2. 2 Contoh Perhitungan Skor *Hybrid*

| Kondisi                              | Skor CB | Skor CF        | Skor hybrid ( $\alpha$ )                   |
|--------------------------------------|---------|----------------|--------------------------------------------|
| <i>Warm-start</i> ( $\alpha = 0.6$ ) | 0.80    | 0.60           | $0.6 \times 0.80 + 0.4 \times 0.60 = 0.72$ |
| <i>Cold-start</i> ( $\alpha = 1$ )   | 0.80    | Tidak tersedia | $1 \times 0.80 + 0 \times 0 = 0.80$        |

Pada kondisi *cold-start*, Skor\_CF belum dapat dihitung karena ketiadaan data interaksi, sehingga mekanisme *switching* menetapkan  $\alpha = 1$  dan skor akhir sepenuhnya

berasal dari *content-based filtering*. Contoh ini menegaskan bahwa formula pada persamaan (2.1) berlaku konsisten pada kedua kondisi, dengan  $\alpha$  sebagai parameter yang menyesuaikan secara otomatis. Mekanisme otomatis ini menghilangkan kebutuhan intervensi manual untuk menentukan kapan sistem harus beralih strategi, sehingga proses rekomendasi tetap berjalan mulus dari sudut pandang pengguna meskipun terjadi perubahan pendekatan di baliknya. Konsistensi ini menjadi salah satu keunggulan utama pendekatan *switching-weighted* dibandingkan penggunaan satu teknik tunggal untuk seluruh kondisi pengguna. Keunggulan ini memastikan sistem tetap dapat memberikan rekomendasi yang bermakna tanpa memerlukan penyesuaian manual di setiap kondisi pengguna yang berbeda.

Pengembangan terkini dalam sistem rekomendasi *hybrid* terus berlanjut untuk menangani tantangan seperti *cold-start*, bias, dan dinamisnya preferensi pengguna, sebagaimana ditunjukkan pada penelitian terdahulu yang telah dibahas, meskipun demikian, kombinasi *content-based* dan *collaborative filtering* klasik yang menjadi fokus penelitian ini tetap menjadi fondasi yang relevan untuk domain spesifik seperti pencarian kos. Berbagai pendekatan lanjutan seperti pemanfaatan graf pengetahuan, *model* berbasis *deep learning*, maupun *large language model* memang menunjukkan peningkatan akurasi pada skala data besar, namun umumnya memerlukan volume data dan sumber daya komputasi yang jauh lebih besar dibandingkan skala prototipe pada penelitian ini. Oleh karena itu, kombinasi *content-based* dan *collaborative filtering* klasik dipandang lebih sesuai untuk konteks penelitian yang menekankan validitas konsep dan kemudahan implementasi pada aplikasi *mobile*. Pendekatan ini juga memudahkan interpretasi hasil rekomendasi, karena kontribusi masing-masing komponen terhadap skor akhir dapat ditelusuri secara langsung melalui persamaan. Kesederhanaan ini turut mendukung transparansi sistem, sehingga admin maupun peneliti dapat memahami alasan di balik setiap rekomendasi yang dihasilkan.

#### **2.2.4 Cosine Similarity**

*Model hybrid filtering* yang digunakan pada penelitian ini bekerja melalui alur berikut. Profil penyewa dan atribut kos diproses oleh komponen *content-based filtering* menggunakan *cosine similarity* [15], [19], sedangkan data interaksi pengguna–kos diproses secara terpisah oleh komponen *collaborative filtering* berbasis memori [15], [16]. Kedua komponen menghasilkan skor kemiripan masing-masing yang mencerminkan relevansi kos terhadap pengguna dari sudut pandang yang berbeda. Kedua skor tersebut selanjutnya digabungkan menggunakan strategi *weighted hybrid filtering*

sebagaimana dijelaskan pada Subbab 2.2.3 [6], [8], [15], sebelum diurutkan (*ranking*) untuk menghasilkan daftar *Top-N* kos yang direkomendasikan [15]. Alur ini menegaskan bahwa kedua komponen bekerja secara paralel dan saling melengkapi, bukan saling menggantikan, sehingga kelemahan salah satu komponen dapat ditutupi oleh komponen lainnya [6], [8], [15].

### 2.3 Cold-Start

*Cold-start* terjadi ketika sistem belum memiliki cukup data interaksi untuk pengguna atau item baru, sehingga *collaborative filtering* tidak dapat menghitung kemiripan secara andal. Kondisi ini berlawanan dengan *warm-start*, yaitu kondisi saat pengguna atau item telah memiliki riwayat interaksi yang memadai sehingga *collaborative filtering* dapat bekerja secara optimal. Tinjauan sistematis Yuan dan Hernandez membedakan *user cold-start* dan *item cold-start* serta mengelompokkan solusinya menjadi pendekatan berbasis data dan berbasis metode [11]. Perbedaan performa antara kondisi *warm-start* dan *cold-start* inilah yang menjadi salah satu fokus pengujian pada penelitian ini. Dengan membandingkan performa pada kedua kondisi tersebut, penelitian ini dapat menilai seberapa efektif mekanisme *user profiling* yang diusulkan dalam menutup kesenjangan performa yang biasanya muncul saat sistem belum memiliki data interaksi yang memadai.

### 2.4 User Profiling

*User profiling* adalah proses membangun representasi karakteristik pengguna, seperti demografi, preferensi, dan konteks, untuk mempersonalisasi rekomendasi [20]. Profil pengguna dapat menjadi acuan awal ketika riwayat interaksi belum tersedia, sehingga berperan penting dalam penanganan *cold-start*; penggabungan data profil, demografis, dan lokasi terbukti efektif mengatasinya [14], [16]. Pemodelan profil yang baik juga berkontribusi pada aspek keadilan dan kualitas rekomendasi di luar akurasi [21]. Pada penelitian ini, profil pengguna dirancang untuk mencakup atribut yang relevan dengan konteks pencarian kos, seperti status sebagai mahasiswa atau pekerja, anggaran, dan preferensi lokasi, sehingga informasi yang terkumpul benar-benar dapat dimanfaatkan oleh komponen *content-based filtering*. Aspek keadilan ini penting agar sistem tidak secara sistematis mengabaikan kelompok pengguna tertentu hanya karena keterbatasan data interaksi mereka.

Dalam penelitian ini, profil pengguna dimanfaatkan sebagai sumber informasi awal ketika pengguna baru pertama kali menggunakan aplikasi. Sebagai ilustrasi, pengguna yang mengisi profil sebagai mahasiswa dengan anggaran terbatas dan preferensi dekat kampus akan langsung menerima rekomendasi kos dengan harga terjangkau dan jarak

dekat kampus, meskipun pengguna tersebut belum pernah berinteraksi dengan *listing* manapun. Informasi profil tersebut digunakan untuk membangun preferensi awal sehingga sistem tetap dapat memberikan rekomendasi meskipun pengguna belum memiliki riwayat interaksi dengan kos yang tersedia. Ilustrasi ini menunjukkan bahwa profil pengguna berfungsi sebagai pengganti sementara riwayat interaksi, memungkinkan sistem tetap relevan sejak momen pertama pengguna berinteraksi dengan aplikasi. Contoh ini menggambarkan bagaimana informasi profil yang sederhana sekalipun dapat langsung diterjemahkan menjadi rekomendasi yang bermakna bagi pengguna baru.

Pemanfaatan profil dan karakteristik pengguna terbukti meningkatkan relevansi rekomendasi dan efektif untuk mengatasi *cold-start* [20], sehingga profil pengguna yang diisi sejak awal pendaftaran menjadi dasar yang memadai bagi mekanisme *cold-start* pada penelitian ini. Pendekatan ini menghindari ketergantungan pada data eksternal seperti hasil survei terpisah, karena seluruh informasi yang dibutuhkan sudah tersedia begitu pengguna menyelesaikan proses registrasi. Dengan demikian, tidak ada jeda waktu antara pendaftaran pengguna baru dan ketersediaan rekomendasi awal yang dipersonalisasi. Ketiadaan jeda ini menjadi salah satu keunggulan utama pendekatan yang diusulkan dibandingkan metode yang bergantung pada pengumpulan data preferensi secara terpisah., sehingga proses pengumpulan data profil terintegrasi langsung dengan alur penggunaan aplikasi

Mekanisme penanganan *cold-start* bagi pengguna baru pada penelitian ini bekerja secara bertahap yaitu saat pengguna baru mendaftar dan mengisi profil, sistem memeriksa ketersediaan riwayat interaksi sebelum menentukan pendekatan yang digunakan, jika riwayat interaksi belum tersedia, sistem sepenuhnya mengandalkan *content-based filtering* berbasis profil untuk menghasilkan rekomendasi awal, sesuai dengan strategi *switching* yang dijelaskan pada subbab 2.2.3. Seiring pengguna mulai berinteraksi dengan sistem, seperti memberikan penilaian atau menyimpan kos, komponen *collaborative filtering* mulai memperoleh data yang cukup untuk berkontribusi, sehingga sistem beralih menggunakan pendekatan *hybrid* sepenuhnya. Alur ini menunjukkan bahwa penanganan *cold-start* pada penelitian ini bersifat progresif, bukan sekadar solusi sementara yang diabaikan setelah pengguna aktif. Transisi bertahap ini memastikan bahwa kualitas rekomendasi terus meningkat seiring bertambahnya data interaksi, tanpa mengharuskan pengguna melalui proses onboarding tambahan atau perubahan antarmuka yang mencolok. Dari sudut pandang pengguna, transisi ini berlangsung mulus tanpa disadari,

karena antarmuka aplikasi tetap konsisten meskipun mekanisme di baliknya berubah secara bertahap.

## 2.5 Metrik Evaluasi

Kualitas sistem rekomendasi *top-N* diukur dengan metrik akurasi peringkat, bukan sekadar ketepatan prediksi rating. Empat metrik utama yang digunakan pada penelitian ini dijelaskan sebagai berikut [18]. Empat metrik ini dipilih karena secara khusus mengukur kualitas urutan rekomendasi, bukan hanya ketepatan prediksi nilai rating secara individual.

- *Precision@K* mengukur proporsi item relevan di antara K item teratas yang direkomendasikan:  $Precision@K = (\text{jumlah item relevan pada K teratas}) / K$ .
- *Recall@K* mengukur proporsi item relevan yang berhasil ditemukan di antara seluruh item relevan yang tersedia:  $Recall@K = (\text{jumlah item relevan pada K teratas}) / (\text{total item relevan})$ .
- *Normalized Discounted Cumulative Gain (NDCG)* mengukur kualitas urutan rekomendasi dengan memberi bobot lebih tinggi pada item relevan yang muncul di posisi lebih atas.
- *Mean Average Precision (MAP)* menghitung rata-rata *Average Precision* dari seluruh pengguna uji, sehingga mencerminkan kualitas peringkat secara menyeluruh, bukan hanya pada satu posisi K tertentu.

Evaluasi dilakukan secara *offline* dengan pembagian data train/test, serta dibandingkan terhadap metode *baseline* seperti popularitas, *content-based filtering* murni, dan *collaborative filtering* murni. Tren terbaru pada bidang ini juga menunjukkan pemanfaatan *deep learning* [3], representasi graf untuk rekomendasi berbasis lokasi [22], serta *Large Language Model* untuk memperkaya pemahaman konten dan mengatasi *sparsity* [23]. Pada penelitian ini, evaluasi difokuskan pada metode klasik yang telah disebutkan sebelumnya, mengingat skala data prototipe belum memungkinkan penerapan pendekatan berbasis *deep learning* maupun *LLM* secara optimal. Keterbatasan ini konsisten dengan cakupan penelitian yang menekankan validitas konsep pada skala prototipe, bukan optimasi performa pada skala data besar. Perkembangan ini menunjukkan bahwa bidang sistem rekomendasi terus berkembang pesat, meskipun penelitian ini memilih untuk berfokus pada pendekatan klasik yang lebih sesuai dengan skala data yang tersedia.

Selain metrik akurasi peringkat, penelitian ini turut mengukur usabilitas prototipe antarmuka menggunakan *System Usability Scale* (SUS), instrumen kuesioner sepuluh butir yang menghasilkan skor 0–100 untuk menilai kemudahan penggunaan sistem dari perspektif pengguna. Skor SUS melengkapi metrik akurasi rekomendasi dengan dimensi pengalaman pengguna yang tidak tertangkap oleh *Precision@K*, *Recall@K*, *NDCG*, maupun *MAP*. Kombinasi metrik akurasi dan usabilitas ini memungkinkan penelitian untuk menilai keberhasilan sistem secara lebih menyeluruh, baik dari sisi ketepatan algoritmik maupun dari sisi pengalaman pengguna yang berinteraksi langsung dengan prototipe. Penilaian yang menyeluruh ini penting karena sistem yang akurat secara algoritmik belum tentu dirasakan mudah digunakan oleh pengguna sesungguhnya. Skor SUS di atas 68 umumnya dianggap menunjukkan tingkat usabilitas yang baik menurut standar yang berlaku secara luas dalam penelitian usabilitas.

## **2.6 Software Development Life Cycle (SDLC)**

*Software Development Life Cycle* (SDLC) adalah kerangka kerja tahapan yang mengatur proses pengembangan perangkat lunak secara sistematis, mulai dari perencanaan, analisis kebutuhan, perancangan, implementasi, pengujian, hingga pemeliharaan [17]. Setiap *model* SDLC mengatur urutan dan hubungan antartahapan tersebut secara berbeda, sehingga pemilihan *model* yang tepat perlu disesuaikan dengan karakteristik proyek, seperti kejelasan kebutuhan di awal proyek, tingkat keterlibatan pengguna, serta kebutuhan akan fleksibilitas terhadap perubahan [17]. *Model* SDLC yang umum digunakan antara lain *Waterfall* yang bersifat linear dan sekuensial, *Agile* yang bersifat iteratif dan inkremental, serta *Prototyping* yang menekankan pembangunan rancangan awal untuk memperoleh umpan balik pengguna secara dini. Pemilihan *model* SDLC yang sesuai berpengaruh langsung terhadap efisiensi proses pengembangan serta kesesuaian hasil akhir dengan kebutuhan pengguna. Pada penelitian ini, *model Prototyping* dipilih sebagai pendekatan SDLC yang digunakan, sebagaimana diuraikan lebih lanjut pada subbab 2.7.

## **2.7 Prototyping**

*Prototyping* adalah salah satu *model* SDLC yang membangun rancangan awal sistem (prototipe) untuk memperoleh umpan balik dari pengguna sebelum pengembangan sistem final, sehingga memungkinkan penyesuaian kebutuhan secara iteratif [17]. Pendekatan ini berbeda dari *model Waterfall* yang bersifat linear dan kurang fleksibel terhadap perubahan kebutuhan di tengah proses pengembangan [17]. Proses *Prototyping* umumnya dimulai dari pengumpulan kebutuhan awal, dilanjutkan dengan perancangan



dan pembangunan prototipe, evaluasi prototipe oleh pengguna, serta penyempurnaan prototipe secara berulang hingga memenuhi kebutuhan yang diharapkan. Model *Prototyping* sesuai digunakan pada penelitian yang menghasilkan produk berupa prototipe aplikasi, karena memungkinkan penyempurnaan rancangan *model* rekomendasi maupun antarmuka berdasarkan umpan balik pada tiap iterasi pengembangan. Pada penelitian ini, prototipe dikembangkan secara bertahap dimulai dari modul inti *RecommendationService*, dilanjutkan dengan antarmuka aplikasi *Flutter*, dan disempurnakan berdasarkan hasil evaluasi performa *model* pada tiap iterasi.

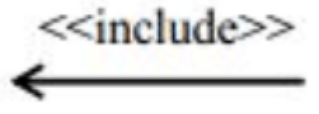
## 2.8 Unified Modeling Language (UML)

*Unified Modeling Language* (UML) merupakan notasi standar untuk memodelkan sistem berorientasi objek [17], [24]. Beberapa diagram yang digunakan dalam penelitian ini diuraikan berikut.

### 2.8.1 Use Case Diagram

*Use case diagram* menggambarkan interaksi antara aktor dan fungsi-fungsi yang disediakan sistem sehingga membantu memetakan kebutuhan fungsional [24]. Diagram ini terdiri atas tiga elemen utama, yaitu aktor yang merepresentasikan pengguna atau sistem eksternal yang berinteraksi dengan sistem, *use case* yang merepresentasikan fungsi atau layanan yang disediakan sistem, serta batas sistem (*system boundary*) yang menunjukkan cakupan sistem yang dimodelkan. Setiap elemen memiliki notasi standar agar diagram dapat dipahami secara konsisten oleh berbagai pihak [24]. Pada penelitian ini, *use case diagram* digunakan untuk memodelkan interaksi antara dua aktor, yaitu pengguna (calon penyewa) yang berinteraksi dengan modul sistem rekomendasi KOSKITA seperti mengisi profil, menerima rekomendasi, dan melihat detail kos, serta admin yang berinteraksi dengan panel pengelolaan data kos. Tabel 2. 3 merangkum notasi *use case diagram* yang digunakan dalam penelitian ini.

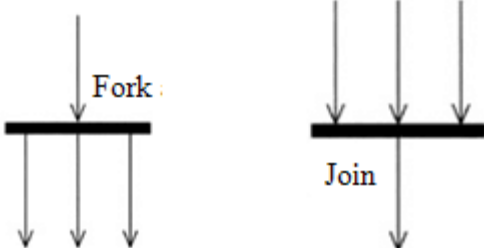
| Notasi                                                                              | Nama            | Keterangan                                                                        |
|-------------------------------------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------|
|  | Aktor           | Merepresentasikan pengguna atau sistem eksternal yang berinteraksi dengan sistem. |
|  | <i>Use Case</i> | Merepresentasikan fungsi atau layanan yang disediakan sistem.                     |
|  | Batas Sistem    | Menunjukkan cakupan sistem yang dimodelkan.                                       |

| Notasi                                                                            | Nama           | Keterangan                                                                     |
|-----------------------------------------------------------------------------------|----------------|--------------------------------------------------------------------------------|
|  | Asosiasi       | Menunjukkan interaksi antara aktor dan <i>use case</i> .                       |
|  | <i>Include</i> | Menunjukkan <i>use case</i> yang selalu disertakan dalam <i>use case</i> lain. |

Tabel 2. 3 Notasi *Use Case Diagram*.

### 2.8.2 Activity Diagram

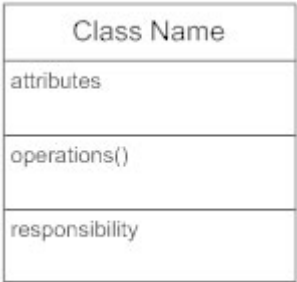
*Activity diagram* menggambarkan alur aktivitas atau proses bisnis, termasuk percabangan dan keputusan, sehingga berguna untuk memodelkan proses pencarian kos yang berjalan saat ini [17]. Diagram ini menggunakan notasi standar yang terdiri atas titik awal (*initial node*), aktivitas (*action*), titik keputusan (*decision node*), garis alur (*control flow*), serta titik akhir (*final node*) untuk merepresentasikan urutan proses secara visual [17]. *Activity diagram* cocok digunakan untuk memodelkan proses yang melibatkan percabangan logika, seperti proses pencarian kos yang melibatkan keputusan pengguna dalam menetapkan *filter*, atau proses penanganan *cold-start* yang bercabang berdasarkan ketersediaan riwayat interaksi pengguna. Pada penelitian ini, *activity diagram* digunakan untuk memodelkan proses bisnis pencarian kos saat ini (Gambar 3. 2) serta perbandingan alur proses pada KOSKITA (Gambar 3. 3). Tabel 2. 4 merangkum notasi *activity diagram* yang digunakan dalam penelitian ini.

| Notasi                                                                              | Nama                   | Keterangan                                                                    |
|-------------------------------------------------------------------------------------|------------------------|-------------------------------------------------------------------------------|
|  | <i>Initial Node</i>    | Menandai titik awal aktivitas.                                                |
|  | <i>Action/Activity</i> | Merepresentasikan satu aktivitas atau langkah proses.                         |
|  | <i>Decision Node</i>   | Menunjukkan titik percabangan berdasarkan kondisi tertentu.                   |
|  | <i>Control Flow</i>    | Menunjukkan urutan aliran aktivitas.                                          |
|  | <i>Final Node</i>      | Menandai titik akhir aktivitas.                                               |
|  | <i>Fork/Join</i>       | Menunjukkan aktivitas paralel yang berjalan bersamaan atau bergabung kembali. |

Tabel 2. 4 Notasi *Activity Diagram*.

### 2.8.3 Class Diagram

*Class diagram* menggambarkan struktur statis sistem berupa kelas, atribut, dan relasinya, yang menjadi dasar perancangan basis data dan *model* rekomendasi [25]. Setiap kelas direpresentasikan sebagai kotak yang terbagi menjadi tiga bagian, yaitu nama kelas, atribut, dan operasi (*method*), sedangkan hubungan antarkelas digambarkan melalui garis relasi seperti asosiasi, agregasi, komposisi, dan pewarisan (*inheritance*) [24]. *Class diagram* memberikan definisi formal terhadap struktur data yang akan diimplementasikan dalam basis data maupun objek pemrograman, sehingga penting untuk memastikan konsistensi antara rancangan dan implementasi [24]. Pada penelitian ini, *class diagram* akan digunakan pada Bab IV untuk merancang struktur kelas yang merepresentasikan entitas seperti Pengguna, Admin, Kos, Profil, dan Interaksi, beserta relasinya dengan modul *RecommendationService*. Tabel 2. 5 merangkum notasi *class diagram* yang digunakan dalam penelitian ini.

| Notasi                                                                              | Nama          | Keterangan                                                                      |
|-------------------------------------------------------------------------------------|---------------|---------------------------------------------------------------------------------|
|   | Kelas         | Merepresentasikan entitas beserta atribut dan operasinya.                       |
|  | Asosiasi      | Menunjukkan hubungan antarkelas.                                                |
|  | Agregasi      | Menunjukkan hubungan “memiliki” yang longgar (bagian dapat berdiri sendiri).    |
|  | Komposisi     | Menunjukkan hubungan “memiliki” yang kuat (bagian tidak dapat berdiri sendiri). |
|  | Generalisasi  | Menunjukkan hubungan turunan antarkelas.                                        |
|  | Multiplisitas | Menunjukkan jumlah keterkaitan antarobjek.                                      |

Tabel 2. 5 Notasi *Class Diagram*.

## 2.9 Teknologi Pengembangan

Bagian ini menguraikan teknologi yang digunakan untuk mengembangkan prototipe KOSKITA, baik pada sisi aplikasi pengguna maupun sisi panel admin. Sesuai

arahan dosen pembimbing, prototipe KOSKITA memerlukan peran admin yang beroperasi untuk mengelola data kos, memoderasi konten, dan memantau aktivitas pengguna, sehingga sistem tidak hanya terdiri atas aplikasi *mobile* bagi calon penyewa, sehingga teknologi yang diuraikan mencakup kerangka kerja dan bahasa pemrograman untuk aplikasi *mobile* (*Flutter* dan *Dart*), kerangka kerja untuk panel admin berbasis web (*Laravel*), basis data sebagai lapisan penyimpanan data, serta arsitektur konseptual yang menyatukan seluruh komponen tersebut. Pemisahan ini memungkinkan pengguna dan admin berinteraksi dengan sistem melalui antarmuka yang sesuai dengan kebutuhan masing-masing peran, tanpa mengorbankan konsistensi data di antara keduanya.

### 2.9.1 *Flutter* dan *Dart*

*Flutter* adalah kerangka kerja (*framework*) *sumber terbuka* yang dikembangkan oleh Google untuk membangun aplikasi lintas platform (*cross-platform*) menggunakan satu basis kode yang dikompilasi langsung menjadi kode *native* untuk Android maupun iOS, tanpa melalui lapisan penerjemah (*bridge*) seperti pada beberapa kerangka kerja *hybrid* lainnya [26]. Karakteristik ini membuat performa aplikasi *Flutter* relatif setara dengan aplikasi *native*, sekaligus mempercepat proses pengembangan karena satu basis kode dapat digunakan untuk berbagai platform [27]. Karakteristik ini menjadikan *Flutter* pilihan yang sesuai untuk penelitian ini, mengingat pengembangan prototipe dengan sumber daya terbatas memerlukan efisiensi tanpa mengorbankan kualitas tampilan maupun performa aplikasi pada perangkat pengguna. Efisiensi ini memungkinkan peneliti untuk lebih memfokuskan waktu pengembangan pada logika rekomendasi dibandingkan penyesuaian antarmuka untuk masing-masing platform secara terpisah. Karakteristik kompilasi langsung ke kode *native* inilah yang membedakan *Flutter* dari kerangka kerja *hybrid* berbasis *WebView* yang cenderung memiliki performa lebih rendah.

*Dart* adalah bahasa pemrograman yang digunakan oleh *Flutter*, bersifat berorientasi objek dan mendukung baik kompilasi *Just-In-Time* (JIT) yang memungkinkan fitur *hot reload* selama pengembangan, maupun kompilasi *Ahead-Of-Time* (AOT) yang menghasilkan kode *native* untuk performa optimal saat aplikasi dijalankan. Pada penelitian ini, *Dart* digunakan untuk membangun antarmuka aplikasi KOSKITA yang mengonsumsi hasil rekomendasi dari *REST API*, sedangkan seluruh mesin rekomendasi (*RecommendationService*), termasuk perhitungan *cosine similarity* dan penggabungan skor *hybrid*, diimplementasikan di sisi *backend* menggunakan *Laravel* sebagaimana dijelaskan pada subbab 2.9.2 dan 2.9.9, sehingga aplikasi *mobile* berperan sebagai lapisan antarmuka (*thin client*) tanpa memuat logika perhitungan rekomendasi. Pemisahan ini

memastikan bahwa perubahan pada algoritma rekomendasi di sisi *backend* tidak memerlukan pembaruan aplikasi *mobile* secara langsung, sehingga proses pemeliharaan dan penyempurnaan *model* dapat dilakukan secara independen dari siklus rilis aplikasi.

Perlu ditegaskan bahwa “*Switching-Weighted Combiner*” yang disebutkan pada subbab 2.9.9 bukan merupakan teknik atau istilah yang berbeda dari *hybrid filtering*. *Switching-Weighted Combiner* adalah nama modul perangkat lunak yang mengimplementasikan strategi *switching hybrid* berkomponen *weighted* yang telah dijelaskan secara teoretis pada subbab 2.2.3, yaitu penggabungan skor *content-based filtering* dan *collaborative filtering* menggunakan mekanisme *switching* untuk menentukan komponen yang aktif, serta mekanisme *weighted* untuk mengatur proporsi kontribusi masing-masing komponen. Dengan kata lain, *hybrid filtering* adalah nama teknik atau metode pada tataran teori, sedangkan *Switching-Weighted Combiner* adalah nama kelas atau modul yang merealisasikan teknik tersebut dalam kode program. Perbedaan penamaan ini sengaja ditegaskan untuk menghindari kesan bahwa penelitian ini menggunakan dua pendekatan berbeda, padahal keduanya merujuk pada konsep yang sama pada tingkat abstraksi yang berbeda. Kejelasan istilah ini penting agar pembaca tidak salah menafsirkan struktur kode sebagai indikasi adanya metode tambahan di luar *hybrid filtering* yang telah dijelaskan sebelumnya.

### 2.9.2 *Laravel*

*Laravel* adalah kerangka kerja (*framework*) *sumber terbuka* berbasis bahasa pemrograman PHP yang dirancang untuk mempercepat pengembangan aplikasi web dengan menyediakan struktur kode yang rapi, konvensi penamaan yang konsisten, serta berbagai komponen siap pakai seperti sistem *routing*, *middleware*, dan mekanisme autentikasi pengguna [28]. Kerangka kerja ini menerapkan pola arsitektur *Model-View-Controller* (MVC) yang memisahkan logika data (*model*), tampilan antarmuka (*view*), dan logika pemrosesan permintaan (*controller*), sehingga kode aplikasi menjadi lebih terorganisir dan mudah dipelihara seiring bertambahnya fitur [28]. *Laravel* juga menyediakan *Blade*, mesin *templating* bawaan yang memungkinkan penyusunan tampilan halaman web secara modular dan dapat digunakan ulang antarhalaman [28]. Pada penelitian ini, *Laravel* dipilih sebagai kerangka kerja untuk membangun panel admin berbasis web yang digunakan oleh pengelola KOSKITA untuk mengelola data kos, memoderasi konten, dan memantau aktivitas pengguna, sehingga terpisah dari aplikasi *mobile* yang dibangun dengan *Flutter* namun tetap terhubung pada basis data yang sama. Pemilihan *Laravel* didasarkan pada kematangan ekosistemnya, dokumentasi yang

lengkap, serta dukungan komunitas pengembang yang luas, sehingga sesuai untuk pengembangan prototipe panel admin dalam lingkup tugas akhir ini.

Salah satu komponen utama *Laravel* yang dimanfaatkan pada penelitian ini adalah *Eloquent Object-Relational Mapping* (ORM), yaitu lapisan abstraksi yang memungkinkan interaksi dengan basis data dilakukan melalui objek dan *method* PHP tanpa perlu menulis kueri SQL secara manual [28]. Setiap tabel pada basis data direpresentasikan sebagai satu kelas *model*, sedangkan setiap baris data direpresentasikan sebagai satu objek dari kelas tersebut, sehingga operasi seperti menambah, membaca, memperbarui, dan menghapus data kos maupun data pengguna dapat dilakukan dengan sintaks yang ringkas dan konsisten. Eloquent ORM juga mendukung relasi antartabel, seperti relasi satu-ke-banyak antara data kos dan data fasilitas, maupun relasi banyak-ke-banyak antara pengguna dan riwayat interaksi, yang didefinisikan secara deklaratif pada tingkat *model*. Dukungan ini mempermudah admin dalam mengelola data kos yang kompleks tanpa mengorbankan konsistensi struktur data yang dipakai bersama oleh mesin rekomendasi, sehingga panel admin berbasis *Laravel* dan aplikasi *mobile* berbasis *Flutter* dapat beroperasi terhadap sumber data yang sama secara konsisten. Konsistensi ini menjadi krusial mengingat kedua komponen sistem, yaitu panel admin dan aplikasi *mobile*, harus selalu bekerja di atas data kos dan interaksi pengguna yang sama agar rekomendasi yang dihasilkan senantiasa mencerminkan kondisi terkini.

### 2.9.3 REST API

*Representational State Transfer* (REST) adalah gaya arsitektur perangkat lunak yang mengatur komunikasi antarsistem melalui protokol HTTP, dengan setiap sumber daya (*resource*) diakses menggunakan URL tertentu dan dimanipulasi melalui metode standar seperti GET, POST, PUT, dan DELETE [29]. Format data yang umum digunakan dalam pertukaran *REST API* adalah *JavaScript Object Notation* (JSON), yang bersifat ringan dan mudah diuraikan oleh berbagai bahasa pemrograman, termasuk Dart pada sisi klien maupun PHP pada sisi server [29]. Salah satu karakteristik REST adalah sifatnya yang *stateless*, artinya setiap permintaan dari klien harus memuat seluruh informasi yang diperlukan server untuk memprosesnya tanpa bergantung pada sesi sebelumnya, sehingga sesuai untuk komunikasi antara aplikasi *mobile* dan *backend* yang berjalan secara terpisah. Pada penelitian ini, *REST API* menjadi penghubung utama antara aplikasi *mobile* *Flutter* dan *backend* *Laravel*, menggantikan kebutuhan aplikasi *mobile* untuk mengakses basis data secara langsung. Dengan pendekatan ini, kredensial basis data tidak pernah

beredar di sisi klien, sehingga keamanan data lebih terjaga dibandingkan apabila aplikasi *mobile* terhubung langsung ke basis data.

Autentikasi pada *REST API* penelitian ini menggunakan *Laravel Sanctum*, yaitu paket autentikasi ringan bawaan *Laravel* yang menyediakan token akses (*personal access token*) khusus untuk aplikasi *mobile* atau *single-page application* yang berkomunikasi dengan *backend Laravel* [28]. Setiap kali pengguna berhasil login atau mendaftar, *Laravel Sanctum* menerbitkan token yang kemudian disertakan aplikasi *Flutter* pada *header* setiap permintaan berikutnya, sehingga server dapat memverifikasi identitas pengguna tanpa memerlukan autentikasi berbasis sesi *cookie* seperti pada aplikasi web tradisional [28]. Pada penelitian ini, *REST API* menyediakan sejumlah *endpoint*, antara lain untuk registrasi dan autentikasi pengguna, pengambilan data profil, permintaan daftar rekomendasi kos, serta pengiriman data interaksi pengguna kembali ke server. Setiap *endpoint* mengembalikan data dalam format JSON yang kemudian diuraikan oleh aplikasi *Flutter* untuk ditampilkan pada antarmuka pengguna, sesuai dengan peran aplikasi *mobile* sebagai *thin client* sebagaimana dijelaskan pada subbab 2.9.1. Dengan demikian, seluruh proses pengolahan data dan perhitungan rekomendasi tetap berada di sisi *backend Laravel*, sedangkan *REST API* berperan murni sebagai jembatan komunikasi data antara kedua komponen sistem.

#### 2.9.4 HTML

*HyperText Markup Language* (HTML) adalah bahasa markah standar yang digunakan untuk menyusun struktur dan konten halaman web, seperti teks, gambar, tabel, dan formulir, melalui elemen-elemen yang direpresentasikan dalam bentuk *tag* [29]. Setiap elemen HTML memiliki peran semantik tertentu, misalnya elemen tabel untuk menampilkan data terstruktur atau elemen formulir untuk menerima masukan dari pengguna, sehingga struktur dokumen dapat dipahami baik oleh peramban maupun oleh alat bantu aksesibilitas [29]. HTML5 juga memperkenalkan elemen-elemen semantik seperti *header*, *nav*, *main*, dan *footer* yang secara eksplisit menandai peran tiap bagian halaman, sehingga struktur dokumen lebih mudah dipahami dibandingkan penggunaan elemen generik seperti *div* semata [29].

Pada penelitian ini, HTML digunakan untuk menyusun struktur halaman *Blade* pada panel admin KOSKITA, termasuk halaman pengelolaan data kos, moderasi konten, dan pemantauan aktivitas pengguna; elemen semantik HTML5 diterapkan pada struktur halaman tersebut, misalnya elemen tabel untuk menampilkan data kos dan elemen formulir untuk input data. Konsistensi struktur semantik ini turut memudahkan proses

pemeliharaan antarmuka panel admin di masa mendatang, karena setiap bagian halaman memiliki peran yang jelas dan mudah ditelusuri dalam kode.

### 2.9.5 CSS

*Cascading Style Sheets* (CSS) adalah bahasa yang digunakan untuk mengatur tampilan visual elemen HTML, mencakup tata letak, warna, tipografi, serta respons tampilan terhadap berbagai ukuran layar (*responsive design*) [29]. Pemisahan antara struktur (HTML) dan tampilan (CSS) memungkinkan perubahan desain dilakukan tanpa mengubah struktur data yang mendasarinya, sehingga memudahkan pemeliharaan antarmuka panel admin dalam jangka panjang. Pada sisi CSS, tata letak elemen mengikuti *model* kotak (*box model*) yang terdiri atas konten, *padding*, *border*, dan *margin*, sedangkan pemilih (*selector*) seperti *class* dan *id* memungkinkan penerapan gaya secara selektif pada elemen tertentu [29].

Pada penelitian ini, CSS digunakan untuk menyusun tampilan halaman *Blade* panel admin KOSKITA, dengan pengaturan *box model* CSS digunakan untuk menata jarak antarkomponen pada tampilan tabel data dan formulir moderasi. Kemudahan pemeliharaan ini menjadi pertimbangan penting mengingat panel admin akan terus dikembangkan seiring bertambahnya fitur pengelolaan KOSKITA.

### 2.9.6 JavaScript

*JavaScript* adalah bahasa pemrograman berbasis skrip yang berjalan pada sisi klien (*client-side*) di dalam peramban web, digunakan untuk menambahkan interaktivitas pada halaman web tanpa memerlukan pemuatan ulang halaman secara penuh [29]. Melalui manipulasi *Document Object Model* (DOM), *JavaScript* memungkinkan elemen HTML diperbarui secara dinamis sebagai respons terhadap aksi pengguna, seperti validasi formulir sebelum data dikirim ke server atau pembaruan tampilan tabel data tanpa memuat ulang seluruh halaman [29]. *JavaScript* juga dapat berkomunikasi dengan *backend Laravel* melalui permintaan asinkron (*AJAX* atau *fetch API*), sehingga operasi seperti pencarian data kos atau pembaruan status moderasi dapat dilakukan secara responsif. Pada penelitian ini, *JavaScript* digunakan secara terbatas pada panel admin untuk keperluan interaktivitas dasar, seperti validasi formulir input data kos dan konfirmasi sebelum penghapusan data, tanpa menggunakan kerangka kerja *JavaScript* tambahan di luar yang sudah disediakan *Laravel* dan *Bootstrap*. Pendekatan ini dipilih agar kompleksitas pengembangan panel admin tetap sesuai dengan cakupan tugas akhir.



Selain manipulasi DOM, *JavaScript* menyediakan konsep fungsi (*function*) dan penanganan peristiwa (*event handler*) yang memungkinkan kode dijalankan sebagai respons terhadap peristiwa tertentu, seperti klik tombol atau pengiriman formulir, tanpa memblokir eksekusi bagian lain dari halaman [29]. Sifat berbasis peristiwa ini membuat *JavaScript* sesuai untuk validasi input secara langsung di sisi klien sebelum data dikirim ke server, sehingga mengurangi permintaan yang tidak perlu ke *backend Laravel*. Pada penelitian ini, penanganan peristiwa *JavaScript* digunakan pada formulir input data kos di panel admin, misalnya menampilkan pesan peringatan saat kolom wajib tidak diisi atau memunculkan dialog konfirmasi sebelum data kos dihapus, sebelum permintaan dikirim ke server. Penggunaan *JavaScript* secara terbatas ini konsisten dengan keputusan untuk menempatkan seluruh logika rekomendasi di sisi *backend Laravel*, sehingga peran *JavaScript* pada penelitian ini murni untuk meningkatkan interaktivitas antarmuka, bukan untuk komputasi data. Pembatasan peran ini menjaga agar logika bisnis tetap terpusat di satu lokasi, sehingga memudahkan pemeliharaan dan pengujian sistem secara keseluruhan.

### **2.9.7 Bootstrap**

*Bootstrap* adalah kerangka kerja CSS *sumber terbuka* yang menyediakan kumpulan komponen antarmuka siap pakai, seperti tombol, formulir, tabel, navigasi, dan sistem tata letak grid yang responsif terhadap berbagai ukuran layar [30]. Penggunaan *Bootstrap* memungkinkan pengembang menyusun antarmuka yang konsisten dan responsif tanpa perlu menulis seluruh kode CSS dari awal, sehingga mempercepat proses pengembangan antarmuka panel admin [30]. *Bootstrap* dipilih pada penelitian ini karena kompatibel langsung dengan *Blade* tanpa memerlukan alat *build* tambahan seperti *bundler JavaScript* modern, sehingga integrasinya dengan *Laravel* menjadi lebih sederhana untuk skala prototipe. Sistem grid *Bootstrap* juga digunakan untuk menyusun tata letak halaman panel admin, seperti tabel data kos yang dapat difilter dan tabel log aktivitas pengguna, agar tetap dapat diakses dengan baik pada perangkat dengan ukuran layar yang berbeda, sehingga kombinasi *Blade*, HTML, CSS, *JavaScript*, dan *Bootstrap* membentuk lapisan antarmuka panel admin yang terintegrasi dengan kerangka kerja *Laravel* sebagaimana dijelaskan pada subbab 2.9.2. Integrasi teknologi-teknologi ini secara keseluruhan memungkinkan pengembangan antarmuka panel admin dilakukan secara efisien tanpa memerlukan konfigurasi tambahan yang kompleks, sehingga sesuai dengan cakupan dan keterbatasan waktu pengembangan prototipe pada penelitian ini.

Sistem grid *Bootstrap* bekerja melalui kombinasi elemen *container*, *row*, dan *col*, dengan lebar setiap baris dapat dibagi hingga dua belas kolom sehingga tata letak dapat disusun secara fleksibel [30]. *Bootstrap* juga menyediakan titik jeda (*breakpoint*) seperti *sm*, *md*, *lg*, *xl*, dan *xxl* yang menentukan bagaimana kolom menyesuaikan lebarnya pada berbagai ukuran layar, sehingga tampilan tetap rapi mulai dari perangkat *mobile* hingga desktop [30]. Pada penelitian ini, sistem grid dan titik jeda tersebut digunakan untuk menyusun tata letak tabel data kos dan panel *filter* pada panel admin, sehingga kolom data dapat menyesuaikan diri secara otomatis ketika diakses melalui perangkat dengan lebar layar yang berbeda. Kemampuan responsif ini penting mengingat admin KOSKITA berpotensi mengakses panel pengelolaan data kos dari berbagai perangkat, termasuk laptop maupun tablet, sehingga tata letak yang konsisten pada berbagai ukuran layar turut mendukung kenyamanan penggunaan panel admin. Kenyamanan ini secara tidak langsung turut mendukung efisiensi kerja admin dalam mengelola data kos dan memoderasi konten dalam jumlah besar.

### 2.9.8 Basis Data

Basis data (*database*) adalah kumpulan data yang terorganisasi secara sistematis sehingga dapat diakses, dikelola, dan diperbarui secara efisien oleh perangkat lunak maupun penggunaannya [31]. Basis data relasional secara khusus menyimpan data dalam bentuk tabel yang terdiri atas baris dan kolom, dengan setiap tabel merepresentasikan satu entitas, serta hubungan antartabel didefinisikan melalui kunci utama (*primary key*) dan kunci tamu (*foreign key*) [31]. Struktur relasional ini memungkinkan data disimpan tanpa duplikasi berlebih (*normalisasi*), sekaligus mempertahankan integritas hubungan antarentitas ketika data diakses dari berbagai bagian sistem. Pada penelitian ini, basis data relasional digunakan untuk menyimpan seluruh entitas yang dibutuhkan sistem KOSKITA, meliputi data pengguna, data admin, atribut kos, profil preferensi, serta riwayat interaksi yang menjadi masukan bagi mesin rekomendasi. Pemilihan basis data relasional dianggap sesuai karena hubungan antarentitas pada domain kos, seperti satu kos yang memiliki banyak fasilitas atau satu pengguna yang memiliki banyak interaksi, bersifat terstruktur dan dapat dimodelkan secara jelas melalui relasi antartabel.

Basis data yang sama diakses oleh dua modul di sisi backend Laravel, yaitu panel admin yang menggunakan Eloquent ORM untuk operasi data, serta modul *RecommendationService* yang membaca data profil dan interaksi untuk keperluan perhitungan *cosine similarity* dan penggabungan skor *hybrid*. Konsistensi struktur data antara kedua komponen ini penting untuk dijaga, karena data yang diperbarui melalui

panel admin, misalnya penambahan kos baru atau perubahan atribut fasilitas, harus segera tercermin pada hasil rekomendasi yang diterima pengguna. Ketidaksesuaian struktur data antara kedua sisi berpotensi menyebabkan kegagalan pembacaan data atau rekomendasi yang tidak akurat, sehingga skema basis data perlu dirancang secara terpadu sejak awal. Skema basis data dirancang dengan mempertimbangkan kebutuhan kedua sisi tersebut, sehingga tabel-tabel utama seperti kos, pengguna, profil, dan interaksi dapat diakses secara efisien baik untuk operasi pengelolaan data oleh admin maupun untuk pembacaan data berskala besar oleh mesin rekomendasi. Rancangan skema basis data secara rinci, termasuk entitas dan relasinya, akan diuraikan lebih lanjut pada Bab IV melalui *class diagram* sebagaimana dijelaskan pada subbab 2.8.3.

#### **2.9.9 Arsitektur *RecommendationService***

Arsitektur konseptual *RecommendationService* pada penelitian ini disusun dalam tiga lapisan yang seluruhnya berjalan di sisi *backend Laravel*. Lapisan antarmuka pada prototipe aplikasi *Flutter* hanya menampilkan hasil rekomendasi yang diperoleh melalui *REST API*, tanpa melakukan perhitungan apa pun. Lapisan logika, yang berjalan di *Laravel*, berisi modul *content-based filtering*, modul *collaborative filtering* berbasis memori, dan modul penggabung (*combiner*) yang menerapkan strategi *switching hybrid* berkomponen *weighted* sebagaimana dijelaskan pada subbab 2.2.3. Lapisan data menyediakan profil pengguna, atribut kos, dan data interaksi yang menjadi masukan bagi kedua modul filtering tersebut. Pemisahan tiga lapisan ini memudahkan pengujian dan pengembangan tiap komponen secara independen. Penempatan seluruh lapisan logika dan data di sisi *backend* memastikan data interaksi antarpengguna yang dibutuhkan *collaborative filtering* tidak perlu dikirim ke perangkat klien, serta menjaga konsistensi antara *model* yang dievaluasi dan *model* yang digunakan pada aplikasi produksi, sebagaimana dijelaskan pada subbab 2.9.2.

#### **2.10 Penelitian Terdahulu**

Beberapa penelitian terdahulu menjadi rujukan dalam pengembangan sistem ini. Zamanzadeh Darban dan Valipour mengusulkan GHRS, sistem *hybrid* berbasis graf yang menggabungkan demografi dan lokasi serta menunjukkan peningkatan pada *cold-start* [14]. Biswas dan Liu memadukan CF dan *deep learning* untuk menangani *cold-start* melalui profil pengguna [6], sedangkan Zhang dan Ma memanfaatkan *graph attention* untuk rekomendasi berbasis lokasi [22]. Elahi dkk. memperkaya profil pengguna dengan sentimen ulasan [32], dan Yuan serta Hernandez menyediakan landasan teori penanganan *user cold-start* [11]. Pada konteks Indonesia, Rizki dan Rianto menerapkan *hybrid*

*switching* untuk *cold-start* [9], sementara Fitriana dkk. [10] dan Agustian dkk. [13] menerapkan rekomendasi pada domain kos. Ringkasan penelitian terdahulu yang paling relevan disajikan pada Tabel 2. 1.

Tabel 2. 6 Rangkuman Penelitian Terdahulu.

| No | Peneliti (Tahun)                             | Judul Penelitian                                                                               | Jenis Penelitian                                                   | Metode Penelitian                                                                                                                                                       | Hasil Penelitian                                                                                                                                                                                                   | Hyperlink / Sumber                                                                                                                                                   |
|----|----------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | P. K. Biswas & S. Liu (2022)                 | <i>A Hybrid Recommender System for Recommending Smartphones to Prospective Customers</i>       | Penelitian kuantitatif eksperimental.                              | Memadukan <i>collaborative filtering</i> (ALS) dengan <i>deep neural network</i> ; menggabungkan fitur karakteristik, kontekstual, struktural, dan sekuensial pengguna. | <i>Model hybrid</i> mengungguli <i>baseline</i> dan tetap mampu merekomendasikan bagi pengguna baru ( <i>cold-start</i> ) melalui profil. Mendasari penggunaan profil untuk <i>cold-start</i> pada penelitian ini. | <i>Expert Systems with Applications</i> - Scopus Q1.<br>Link:<br><a href="https://doi.org/10.1016/j.eswa.2022.118058">https://doi.org/10.1016/j.eswa.2022.118058</a> |
| 2  | M. Elahi, dkk. (2023)                        | <i>Hybrid Recommendation by Incorporating the Sentiment of Product Reviews</i>                 | Penelitian kuantitatif eksperimental.                              | Mengintegrasikan analisis sentimen ulasan (BERT + reduksi dimensi) ke <i>model deep learning</i> (mis. DeepFM).                                                         | Atribut/konten tambahan memperkaya profil pengguna dan meningkatkan akurasi. Justifikasi penggunaan atribut kaya pada komponen <i>content-based</i> .                                                              | <i>Information Sciences</i> - Scopus Q1.<br>Link:<br><a href="https://doi.org/10.1016/j.ins.2023.01.051">https://doi.org/10.1016/j.ins.2023.01.051</a>               |
| 3  | H. Yuan & A. A. Hernandez (2023)             | <i>User Cold Start Problem in Recommendation Systems: A Systematic Review</i>                  | Studi tinjauan sistematis ( <i>Systematic Literature Review</i> ). | Telaah sistematis literatur 2016-2023 yang difokuskan pada masalah <i>user cold-start</i> .                                                                             | Mengklasifikasikan solusi <i>cold-start</i> serta membedakan <i>user</i> dan <i>item cold-start</i> . Landasan teori utama strategi <i>cold-start</i> berbasis profil.                                             | <i>IEEE Access</i> - Scopus Q1.<br>Link:<br><a href="https://doi.org/10.1109/ACCESS.2023.3338705">https://doi.org/10.1109/ACCESS.2023.3338705</a>                    |
| 4  | Z. Zamanzadeh Darban & M. H. Valipour (2022) | <i>GHRs: Graph-Based Hybrid Recommendation System with Application to Movie Recommendation</i> | Penelitian kuantitatif eksperimental.                              | Graf kesamaan antarpengguna + atribut demografis dan lokasi; <i>autoencoder</i> untuk fitur laten dan <i>clustering</i> pengguna; diuji pada MovieLens.                 | Peningkatan signifikan pada <i>cold-start</i> karena memanfaatkan demografi dan lokasi. Rujukan kunci: profil + lokasi sebagai sinyal <i>cold-start</i> , sejalan dengan profil penyewa dan lokasi kos.            | <i>Expert Systems with Applications</i> - Scopus Q1.<br>Link:<br><a href="https://doi.org/10.1016/j.eswa.2022.116850">https://doi.org/10.1016/j.eswa.2022.116850</a> |
| 5  | J. Zhang & W. Ma (2024)                      | <i>Hybrid Structural Graph Attention Network for POI Recommendation</i>                        | Penelitian kuantitatif eksperimental.                              | <i>Hybrid Structural Graph Attention Network</i> (HS-GAT) atas graf heterogen multisumber (faktor spasial, temporal, sosial).                                           | Struktur graf dan beragam sumber data meningkatkan akurasi rekomendasi lokasi. Contoh <i>hybrid</i> berbasis lokasi yang relevan dengan pemanfaatan lokasi/atribut kos.                                            | <i>Expert Systems with Applications</i> - Scopus Q1.<br>Link:<br><a href="https://doi.org/10.1016/j.eswa.2024.123436">https://doi.org/10.1016/j.eswa.2024.123436</a> |

Berdasarkan penelitian-penelitian tersebut dapat dilihat bahwa *hybrid filtering* telah banyak digunakan untuk meningkatkan kualitas rekomendasi pada berbagai domain. Beberapa penelitian juga menunjukkan bahwa pemanfaatan informasi tambahan seperti profil pengguna mampu membantu mengatasi permasalahan *cold-start*, tetapi penerapan pendekatan tersebut pada sistem rekomendasi kos di Indonesia masih terbatas, sehingga penelitian ini menggabungkan *hybrid filtering* dengan *user profiling* untuk menghasilkan rekomendasi yang lebih sesuai bagi pengguna baru pada domain pencarian kos.

## **BAB III**

### **SISTEM SAAT INI**

#### **3.1 Profil KOSKITA**

KOSKITA adalah aplikasi pencarian dan pemesanan kos yang telah beroperasi untuk pasar Indonesia dengan studi kasus wilayah Tangerang dan sekitarnya. Dari keseluruhan fitur yang telah berjalan, yaitu pencarian, pemesanan, pembayaran, komunikasi, manajemen properti, dan pembukuan, penelitian ini berfokus pada pengembangan modul rekomendasi kos yang belum tersedia beserta prototipe antarmukanya.

##### **3.1.1 Visi**

Menjadi aplikasi pencarian kos yang membantu calon penyewa menemukan hunian yang paling sesuai secara cepat, personal, dan terpercaya. KOSKITA berkomitmen menghadirkan pengalaman pencarian kos yang mudah melalui pemanfaatan teknologi sistem rekomendasi yang cerdas dan adaptif. Aplikasi ini bertujuan memberikan rekomendasi yang relevan berdasarkan kebutuhan, preferensi, dan karakteristik setiap pengguna. Selain itu, KOSKITA berupaya meningkatkan efisiensi proses pencarian kos dengan menyediakan informasi yang akurat, lengkap, dan mudah dipahami. Dengan demikian, KOSKITA diharapkan menjadi platform pencarian kos yang inovatif, terpercaya, dan memberikan nilai tambah bagi calon penyewa maupun pemilik kos.

##### **3.1.2 Misi**

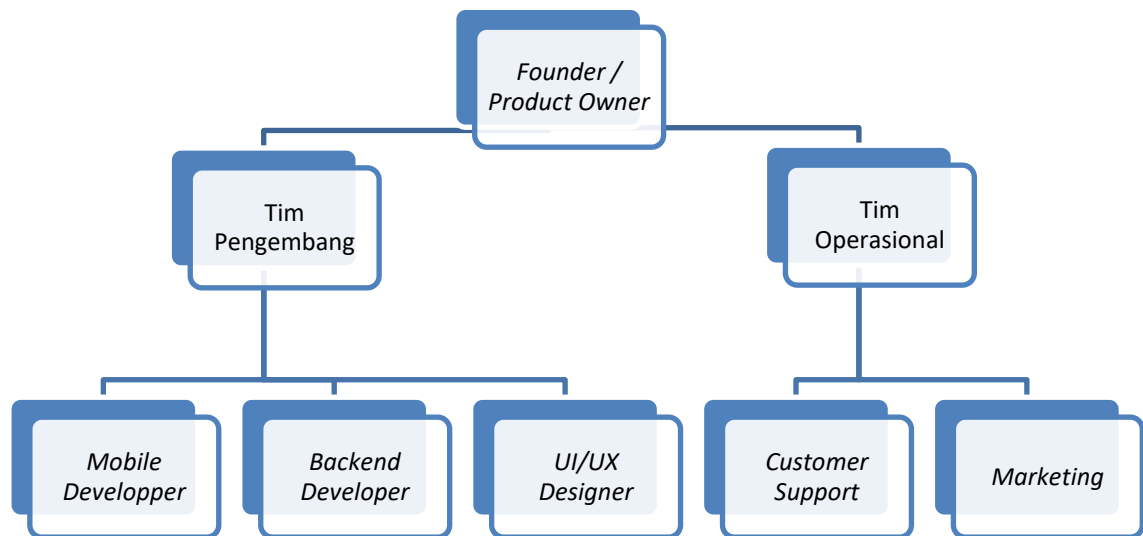
Misi KOSKITA dalam mendukung visi tersebut adalah sebagai berikut.

- Menyediakan rekomendasi kos yang adaptif terhadap profil dan kebutuhan setiap penyewa.
- Mempermudah pengguna baru memperoleh rekomendasi yang relevan meski belum memiliki riwayat interaksi.
- Menyajikan informasi kos yang lengkap dan transparan untuk mendukung pengambilan keputusan.

#### **3.2 Struktur Organisasi**

KOSKITA telah beroperasi sebagai aplikasi pencarian dan pemesanan kos, namun modul sistem rekomendasi yang menjadi fokus penelitian ini masih berada pada tahap pengembangan dan belum diimplementasikan pada aplikasi yang berjalan. Gambar 3. 1 menggambarkan struktur organisasi yang mendukung operasional KOSKITA saat ini

sekaligus pengembangan modul rekomendasi yang sedang berlangsung. Struktur organisasi ini mencerminkan pembagian tanggung jawab antara tim operasional yang menjalankan fitur KOSKITA yang sudah berjalan dan peneliti yang berfokus pada pengembangan modul rekomendasi sebagai kontribusi tugas akhir ini. Kondisi ini menempatkan penelitian pada posisi strategis, karena hasilnya akan langsung diintegrasikan ke dalam platform yang sudah memiliki basis pengguna, bukan sekadar purwarupa yang berdiri sendiri. Pembagian ini memastikan bahwa pengembangan modul rekomendasi dapat berjalan secara terfokus tanpa mengganggu operasional KOSKITA yang sudah berjalan bagi pengguna existing.



Gambar 3. 1 Struktur Organisasi KOSKITA

Pada tahap penelitian ini, peran perancangan dan pengembangan modul sistem rekomendasi serta prototipe antarmukanya dilaksanakan oleh peneliti sebagai bagian dari tugas akhir, sedangkan peran *Backend Developer*, *UI/UX Designer*, *Customer Support*, dan *Marketing* pada struktur tersebut merupakan bagian dari tim operasional KOSKITA yang telah berjalan, di luar cakupan pengembangan modul rekomendasi pada penelitian ini. Pembagian peran ini ditetapkan agar cakupan penelitian tetap terfokus pada perancangan dan implementasi mesin rekomendasi beserta prototipe antarmukanya, tanpa memperluas ke aspek operasional KOSKITA yang berada di luar kendali langsung peneliti. Koordinasi dengan tim operasional tetap dilakukan sejauh diperlukan untuk memastikan modul rekomendasi yang dikembangkan dapat terintegrasi dengan baik pada arsitektur KOSKITA yang sudah ada. Koordinasi ini penting untuk memastikan modul rekomendasi yang dihasilkan benar-benar dapat diintegrasikan ke platform KOSKITA tanpa memerlukan perombakan arsitektur yang signifikan. Pembagian peran ini ditetapkan agar cakupan penelitian tetap terfokus pada perancangan dan implementasi mesin rekomendasi beserta prototipe antarmukanya.

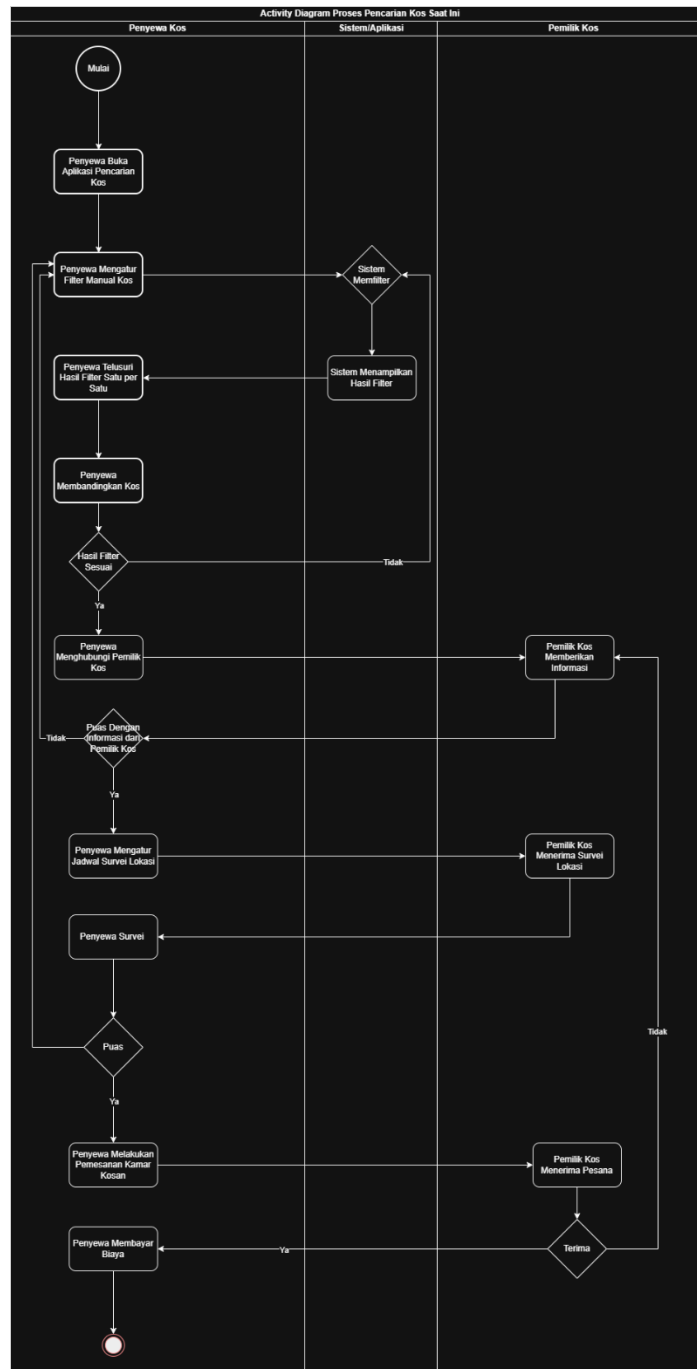


### **3.3 Analisis Sistem Saat Ini**

Bagian ini menganalisis proses bisnis pencarian kos yang berjalan saat ini serta membandingkannya dengan aplikasi sejenis untuk memahami posisi KOSKITA di antara kompetitornya.

#### **3.3.1 Proses Bisnis Pencarian Kos Saat Ini**

Pada aplikasi pencarian kos yang ada, pengguna memulai dengan membuka aplikasi, menetapkan *filter* secara manual (lokasi, harga, tipe, fasilitas), lalu menelusuri daftar *listing* dan membandingkannya satu per satu, apabila belum menemukan kos yang sesuai, pengguna mengubah *filter* dan mengulang proses tersebut hingga menemukan kandidat, kemudian menghubungi pemilik dan melakukan survei sebelum memutuskan. Alur tersebut digambarkan pada Gambar 3. 2. Setiap tahapan dalam alur ini sepenuhnya bergantung pada inisiatif dan kecermatan pengguna, tanpa ada bantuan sistem untuk mempersempit pilihan berdasarkan karakteristik personal pengguna. Ketergantungan penuh pada inisiatif pengguna ini menjadi salah satu akar masalah yang coba diselesaikan melalui pengembangan modul rekomendasi pada penelitian ini.



Gambar 3. 2 Activity Diagram Proses Pencarian Kos Saat Ini

Dari alur tersebut terlihat bahwa beban penyaringan sepenuhnya berada pada pengguna. Tidak ada rekomendasi yang disesuaikan dengan profil pengguna, dan pengguna baru harus memulai dari nol tanpa panduan personal (*cold-start*). Hal ini memperpanjang waktu pencarian dan berisiko menghasilkan keputusan yang kurang optimal. Kondisi ini menegaskan perlunya sistem yang dapat mengambil alih sebagian beban penyaringan tersebut melalui mekanisme rekomendasi yang mempertimbangkan profil dan preferensi masing-masing pengguna secara otomatis. Peralihan beban ini menjadi inti dari kontribusi penelitian, yaitu menggeser peran penyaringan dari sepenuhnya manual menjadi terbantu oleh sistem.

### 3.3.2 Analisis Aplikasi Sejenis

Posisi KOSKITA dipahami melalui perbandingan ketersediaan fitur rekomendasi pada tiga aplikasi kos populer, yaitu Mamikos, Rukita, dan Papikost, terhadap rancangan KOSKITA, sebagaimana disajikan pada Tabel 3. 1. Perbandingan ini mencakup kemampuan pencarian dan penyaringan dasar, ketersediaan rekomendasi berbasis lokasi, serta yang terpenting, ketersediaan rekomendasi yang mempertimbangkan profil individual penyewa dan penanganan terhadap pengguna baru. Perbandingan ini bertujuan untuk menunjukkan secara objektif celah yang masih ada pada aplikasi-aplikasi yang sudah beroperasi, sekaligus memposisikan kontribusi KOSKITA di antara para kompetitornya. Ketiga aplikasi tersebut dipilih karena merupakan platform pencarian kos yang paling banyak digunakan di Indonesia saat ini.

Tabel 3. 1 Perbandingan Fitur Rekomendasi pada Aplikasi Kos.

| Fitur                                                         | Mamikos  | Rukita   | Papikost | KOSKITA          |
|---------------------------------------------------------------|----------|----------|----------|------------------|
| Pencarian berbasis lokasi (kota, kecamatan, kampus, peta)     | Ada      | Ada      | Ada      | Ada              |
| Filter harga (min-maks, bulanan/tahunan, deposit)             | Ada      | Ada      | Ada      | Ada              |
| Filter tipe kos (putra, putri, campur, pasutri, keluarga)     | Ada      | Terbatas | Ada      | Ada              |
| Filter fasilitas (AC, WiFi, KM dalam, dapur, parkir, laundry) | Ada      | Ada      | Ada      | Ada              |
| Filter aturan kos (jam malam, tamu, hewan, merokok)           | Terbatas | Terbatas | Terbatas | Ada              |
| Rekomendasi berbasis lokasi (nearby, dekat kampus/kantor)     | Ada      | Terbatas | Terbatas | Ada              |
| Rekomendasi berbasis profil penyewa                           | Belum    | Belum    | Belum    | Fokus penelitian |
| Penanganan pengguna baru ( <i>cold-start</i> )                | Belum    | Belum    | Belum    | Fokus penelitian |

Tabel 3. 1 menunjukkan bahwa aplikasi yang ada telah unggul pada pencarian dan penyaringan, namun belum menyediakan rekomendasi berbasis profil penyewa maupun penanganan pengguna baru. Penerapan rekomendasi pada domain kos di Indonesia pun masih terbatas dan umumnya menggunakan teknik tunggal seperti *collaborative filtering* [13] atau klasterisasi [10], serta pengukuran kemiripan seperti *cosine similarity* [19]. Hal serupa juga terlihat pada penerapan CF di *e-commerce* [33]. Kondisi ini menegaskan bahwa kesenjangan antara ketersediaan fitur pencarian dasar dan kebutuhan personalisasi berbasis profil masih terbuka lebar, khususnya pada domain kos di Indonesia yang memiliki karakteristik pencarian tersendiri.

Selain belum menyediakan rekomendasi yang dipersonalisasi, aplikasi yang ada masih mengandalkan pengguna untuk menentukan sendiri kriteria pencarian melalui fitur filter. Pendekatan tersebut memang membantu mempersempit pilihan, namun belum

mampu memberikan rekomendasi yang benar-benar disesuaikan dengan karakteristik masing-masing pengguna, terutama bagi pengguna baru yang belum memiliki riwayat aktivitas di dalam sistem. Kondisi tersebut menunjukkan bahwa proses pencarian kos masih berpusat pada upaya pengguna dalam melakukan penyaringan secara manual, sehingga diperlukan sistem yang mampu memberikan rekomendasi secara lebih personal berdasarkan karakteristik masing-masing pengguna. Kebutuhan ini menjadi semakin mendesak mengingat semakin banyaknya pilihan kos yang tersedia, sehingga pengguna memerlukan bantuan sistem untuk menyaring opsi yang paling relevan dengan kebutuhan mereka secara efisien. Kebutuhan ini menjadi semakin mendesak mengingat semakin banyaknya pilihan kos yang tersedia, sehingga pengguna memerlukan bantuan sistem untuk menyaring opsi yang paling relevan dengan kebutuhan mereka secara efisien.

### **3.4 Identifikasi Permasalahan**

Bagian ini mengidentifikasi kendala yang dihadapi pada sistem pencarian kos saat ini berdasarkan hasil analisis pada subbab sebelumnya, yang menjadi dasar perumusan sistem usulan pada penelitian ini.

#### **3.4.1 Kendala Sistem Saat Ini**

Berdasarkan uraian proses bisnis dan analisis aplikasi sejenis pada subbab sebelumnya, teridentifikasi beberapa kendala berikut pada sistem yang berjalan saat ini.

1. Proses pencarian kos masih bertumpu pada penyaringan manual sehingga melelahkan dan rentan menghasilkan keputusan yang kurang optimal, terutama ketika jumlah listing yang tersedia banyak dan atributnya beragam.
2. Belum ada rekomendasi yang disesuaikan dengan profil penyewa, padahal karakteristik penyewa kos sangat beragam, sehingga dua pengguna dengan kebutuhan berbeda menerima hasil pencarian yang sama persis apabila menggunakan filter yang sama.
3. Pengguna baru tidak memperoleh panduan personal karena ketiadaan riwayat interaksi (*cold-start*), sehingga pengalaman pertama menggunakan aplikasi menjadi kurang optimal dan berisiko menurunkan minat pengguna untuk terus menggunakan aplikasi.

## **BAB IV**

### **ANALISIS DAN PERANCANGAN SISTEM USULAN**

#### **4.1 Analisis Kelayakan Sistem**

#### **4.2 Tahap Analisis**

##### **4.2.1 Sistem Usulan (Target Sistem)**

## **BAB V**

### **KESIMPULAN DAN SARAN**

#### **5.1 Kesimpulan**

#### **5.2 Saran**

## DAFTAR PUSTAKA

- [1] M. Arnold, M. Goldschmitt, and T. Rigotti, "Dealing with information overload: A comprehensive review," *Frontiers in Psychology*, vol. 14, art. 1122200, Jun. 2023. doi:10.3389/fpsyg.2023.1122200.
- [2] D. Kowald, D. Yang, and E. Lacic, "Editorial: Reviews in recommender systems: 2022," *Frontiers in Big Data*, vol. 7, art. 1384460, Apr. 2024. doi:10.3389/fdata.2024.1384460.
- [3] H. Zhou, F. Xiong, and H. Chen, "A comprehensive survey of recommender systems based on deep learning," *Applied Sciences*, vol. 13, no. 20, art. 11378, Oct. 2023. doi:10.3390/app132011378.
- [4] L. Jin and M. H. Ryu, "User dissatisfaction and behavioral intention toward personalized advertising recommendation services in Chinese social networking services," *Humanities and Social Sciences Communications*, vol. 12, art. 1857, Dec. 2025. doi:10.1057/s41599-025-06130-x.
- [5] A. Berliani, N. Y. Setiawan, and M. C. Saputra, "Analisis sentimen pada ulasan aplikasi Mamikos di Google Play menggunakan algoritme Support Vector Machine (SVM)," *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer (J-PTIIK)*, vol. 7, no. 9, Sep. 2023.
- [6] P. K. Biswas and S. Liu, "A hybrid recommender system for recommending smartphones to prospective customers," *Expert Systems with Applications*, vol. 208, p. 118058, Dec. 2022. doi:10.1016/j.eswa.2022.118058.
- [7] S. Raza, M. Rahman, S. Kamawal, A. Toroghi, A. Raval, F. Navah, and A. Kazemeini, "A comprehensive review of recommender systems: Transitioning from theory to practice," *Computer Science Review*, vol. 59, art. 100849, 2026. doi:10.1016/j.cosrev.2025.100849.
- [8] V. G. Morales Murillo, D. E. Pinto Avendaño, F. Rojas López, and J. M. Gonzales Calleros, "A systematic literature review on the hybrid approaches for recommender systems," *Computación y Sistemas*, vol. 26, no. 1, pp. 357–372, Aug. 2022. doi:10.13053/cys-26-1-4180.
- [9] M. R. Rizki and Rianto, "Sistem rekomendasi hybrid menggunakan metode switching untuk penanganan cold-start," *Jurnal Teknik Informatika dan Sistem Informasi (JuTISI)*, vol. 10, no. 2, pp. 149–160, Aug. 2024. doi:10.28932/jutisi.v10i2.6220.
- [10] E. Fitriana, W. Windarsyah, and K. Kamarudin, "Clustering kos dengan algoritma K-Means untuk rekomendasi tempat berdasarkan harga dan fasilitas," *Jurnal Media Informatika (JUMIN)*, vol. 6, no. 3, pp. 1990–1995, Jun. 2025. doi:10.55338/jumin.v6i3.6188.
- [11] H. Yuan and A. A. Hernandez, "User cold start problem in recommendation systems: A systematic review," *IEEE Access*, vol. 11, pp. 136958–136977, Dec. 2023. doi:10.1109/ACCESS.2023.3338705.
- [12] K. M. Pandoh, K. Santa, and A. A. Kenap, "Rancang bangun aplikasi penyewaan kost dengan sistem rekomendasi menggunakan content based filtering," *Jurnal Sains Informatika Terapan*, vol. 4, no. 3, pp. 641–651, Nov. 2025.
- [13] A. Agustian, S. P. A. Alkadri, and Istikoma, "Penerapan metode collaborative filtering untuk rekomendasi tempat kos di sekitar kampus UM Pontianak," *Jurnal Informatika Polinema*, vol. 10, no. 3, pp. 333–340, May 2024. doi:10.33795/jip.v10i3.5085.
- [14] Z. Zamanzadeh Darban and M. H. Valipour, "GHRs: Graph-based hybrid recommendation system with application to movie recommendation," *Expert Systems with Applications*, vol. 200, p. 116850, Aug. 2022. doi:10.1016/j.eswa.2022.116850.
- [15] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 3rd ed. Cham: Springer, 2022.

- [16] F. R. Hariri and L. W. Rochim, "Sistem rekomendasi produk aplikasi marketplace berdasarkan karakteristik pembeli menggunakan metode user based collaborative filtering," *Teknika*, vol. 11, no. 3, pp. 208–217, Nov. 2022. doi:10.34148/teknika.v11i3.538.
- [17] A. Dennis, B. H. Wixom, and R. M. Roth, *Systems Analysis and Design*, 8th ed. Hoboken, NJ: Wiley, 2022.
- [18] D. Roy and M. Dutta, "A systematic review and research perspective on recommender systems," *Journal of Big Data*, vol. 9, no. 1, p. 59, May 2022. doi:10.1186/s40537-022-00592-5.
- [19] A. Sanjaya, A. B. Setiawan, U. Mahdiah, I. N. Farida, and A. R. Prasetyo, "Pengukuran kemiripan makna menggunakan cosine similarity dan basis data sinonim kata," *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, vol. 10, no. 4, pp. 747–752, Aug. 2023. doi:10.25126/jtiik.2023106864.
- [20] N. Alqazzaz, M. Alsalman, and F. Alharbi, "Genre-aware user profiling using duration count matrices: A novel approach to enhancing content recommendation systems," *PLOS ONE*, vol. 20, no. 4, p. e0312520, Apr. 2025. doi:10.1371/journal.pone.0312520.
- [21] Y. Deldjoo, D. Jannach, A. Bellogín, A. Difonzo, and D. Zanzonelli, "Fairness in recommender systems: Research landscape and future directions," *User Modeling and User-Adapted Interaction*, vol. 34, no. 1, pp. 59–108, Apr. 2023. doi:10.1007/s11257-023-09364-z.
- [22] J. Zhang and W. Ma, "Hybrid structural graph attention network for POI recommendation," *Expert Systems with Applications*, vol. 248, p. 123436, 2024. doi:10.1016/j.eswa.2024.123436.
- [23] G. E. I. Selim, "Comparative analysis of methodologies and approaches in recommender systems utilizing large language models," *Artificial Intelligence Review*, vol. 58, no. 7, p. 207, 2025. doi:10.1007/s10462-025-11189-8.
- [24] A. Dennis, B. H. Wixom, D. Tegarden, and E. Seeman, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 5th ed. Hoboken, NJ: Wiley, 2015.
- [25] Y. Meng and A. Ban, "Automated UML class diagram generation from textual requirements using NLP techniques," *JOIV: International Journal on Informatics Visualization*, vol. 8, no. 3-2, pp. 1905–1915, Nov. 2024. doi:10.62527/joiv.8.3-2.3482.
- [26] Google, "Flutter documentation," *Flutter*. [Online]. Available: <https://docs.flutter.dev>. Accessed: Jul. 11, 2026.
- [27] S. A. Kinari, N. Funabiki, S. T. Aung, K. H. Wai, M. Mentari, and P. Puspitaningayu, "An independent learning system for Flutter cross-platform mobile programming with code modification problems," *Information*, vol. 15, no. 10, art. 614, Oct. 2024. doi:10.3390/info15100614.
- [28] M. Stauffer, *Laravel: Up & Running*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2023.
- [29] G. Johnson, *Training Guide: Programming in HTML5 with JavaScript and CSS3*. Redmond, WA: Microsoft Press, 2013.
- [30] D. Foreman, *Bootstrap 5 Foundations*. Self-published, 2021.
- [31] S. Sumathi and S. Esakkirajan, *Fundamentals of Relational Database Management Systems*. Berlin: Springer, 2007.
- [32] M. Elahi, D. Khosh Kholgh, M. S. Kiarostami, M. Oussalah, and S. Saghari, "Hybrid recommendation by incorporating the sentiment of product reviews," *Information Sciences*, vol. 625, pp. 738–756, May 2023. doi:10.1016/j.ins.2023.01.051.
- [33] M. Khusnah, R. Gernowo, and B. Surarso, "Implementasi e-commerce dengan sistem informasi rekomendasi menggunakan metode collaborative filtering untuk pengembangan

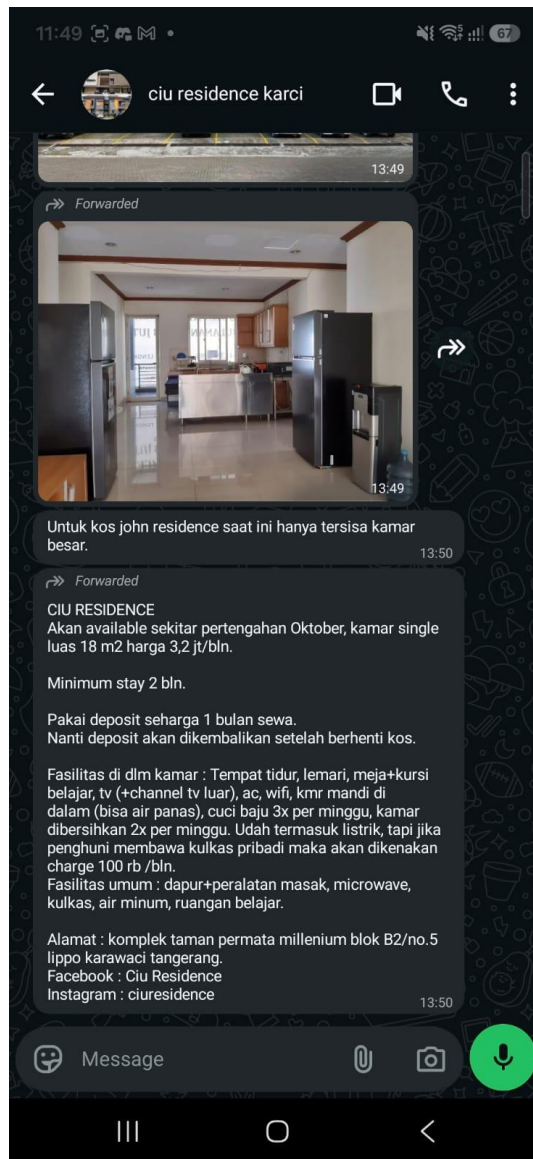


penjualan pada UMKM,” *Jurnal Sistem Informasi Bisnis (JSINBIS)*, vol. 15, no. 1, pp. 134–141, Feb. 2025. doi:10.21456/vol15iss1pp134-141.

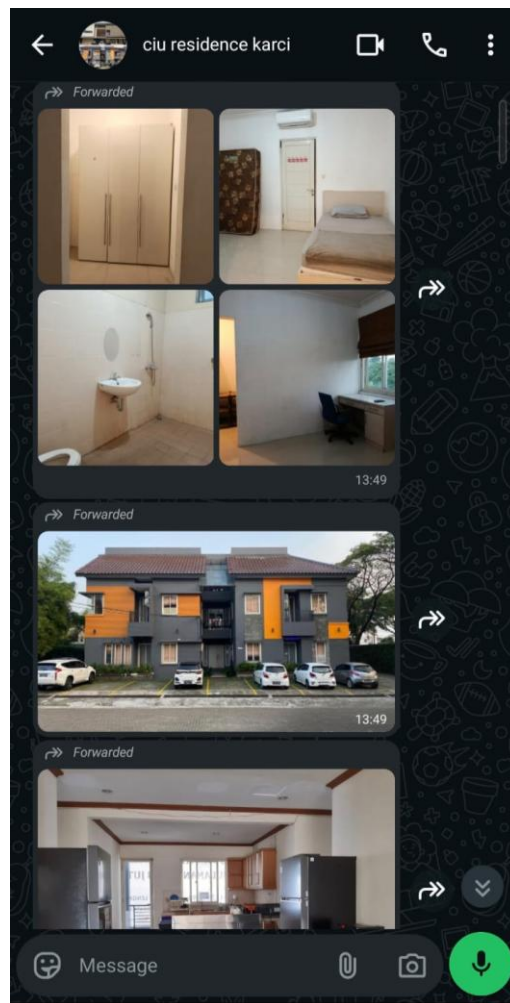
## LAMPIRAN A

### HASIL PERCAKAPAN SEWA KOS

#### A.1 Hasil Percakapan Sewa Kos Melalui Whatsapp



Gambar A.1 Percakapan dengan CIU Residence Karawaci



Gambar A.2 Percakapan Lanjutan dengan CIU Residence Karawaci

## LAMPIRAN B

### LEMBAR MONITORING BIMBINGAN

#### B.1 Lembar Monitoring Bimbingan Tugas Akhir – Dosen Pembimbing (Calandra Alencia Haryani)

|                                                                                   |                                                                                                  |                                                      |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------|
|  | <b>UNIVERSITAS PELITA HARAPAN</b><br>SISTEM PENJAMINAN MUTU INTERNAL                             | No. Dok. : FOR03/PRO07/STA05/SPMI-UPH<br>Revisi : 02 |
|                                                                                   | <b>FORM LEMBAR MONITORING BIMBINGAN TUGAS AKHIR</b><br>FINAL PROJECT SUPERVISION MONITORING FORM | Tanggal : 7 Juni 2026<br>Halaman : 1 dari 1          |
|                                                                                   |                                                                                                  |                                                      |
|                                                                                   |                                                                                                  |                                                      |

|                                       |                         |                                                                                                             |                                                     |
|---------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Nama Mahasiswa<br><i>Student Name</i> | : Bryan Daniel Angelino | Nama Dosen Pembimbing/ Co – Pembimbing<br><i>Supervisor/Lecturer's Name/ Co-Supervising Lecturer's Name</i> | : Calandra Alencia Haryani, S. E.,<br>S.SI., M.T.I. |
| NPM<br><i>Student ID Number</i>       | : 01081230011           |                                                                                                             |                                                     |
| Program Studi<br><i>Study Program</i> | : Sistem Informasi      |                                                                                                             |                                                     |
| Fakultas<br><i>Faculty</i>            | : FAIDAS                |                                                                                                             |                                                     |
| Semester                              | : Akselerasi 2026-2027  |                                                                                                             |                                                     |

| Minggu Ke-<br>Week Number | Hari<br>Day | Tanggal<br>Date | Materi yang didiskusikan<br>Discussed Material/Topics                                                                                                                                           | Tanda tangan Dosen Pembimbing / Co-Pembimbing *)<br>Signature of Supervising Lecturer/Co-Supervising Lecturer *) | Catatan<br>Notes |
|---------------------------|-------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------|
| 1                         | Senin       | 8 Juni 2026     | Batas minimal referensi yang digunakan dan rating referensi harus antara SINTA 1-5 atau Q 1-4 dan kalau conference harus terdeteksi di Scopus. Persyaratan referensi minimal untuk UTS dan UAS. |                               |                  |
| 2                         | Kamis       | 11 Juni 2026    | Memaksimalkan dan memperbaiki tabel penelitian terdahulu.                                                                                                                                       |                               |                  |
| 3                         | Rabu        | 17 Juni 2026    | Revisi proposal mengenai ketepatan judul yang digunakan dan konten proposal.                                                                                                                    |                             |                  |
| 4                         | Senin       | 22 Juni 2026    | Revisi proposal mengenai latar belakang dan metode pengembangan yang digunakan.                                                                                                                 |                             |                  |
| 5                         | Jumat       | 10 Juli 2026    | Revisi Final Bab 1-3 beserta pengecekan ulang konten                                                                                                                                            |                             |                  |
|                           |             |                 |                                                                                                                                                                                                 |                                                                                                                  |                  |

\*) Coret yang tidak perlu  
Cross out what is not necessary.

True Knowledge, Faith in Christ, Godly Character

## B. 2 Lembar Monitoring Bimbingan Tugas Akhir – Dosen Co-Pembimbing (Hery)

|                                                                                   |                                                                                                  |                                                      |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------|
|  | <b>UNIVERSITAS PELITA HARAPAN</b><br>SISTEM PENJAMINAN MUTU INTERNAL                             | No. Dok. : FOR03/PRO07/STA05/SPMI-UPH<br>Revisi : 02 |
|                                                                                   | <b>FORM LEMBAR MONITORING BIMBINGAN TUGAS AKHIR</b><br>FINAL PROJECT SUPERVISION MONITORING FORM | Tanggal : 7 Juni 2026<br>Halaman : 1 dari 1          |
|                                                                                   |                                                                                                  |                                                      |

|                          |                         |                                                                   |                          |
|--------------------------|-------------------------|-------------------------------------------------------------------|--------------------------|
| Nama Mahasiswa           | : Bryan Daniel Angelino | Nama Dosen Pembimbing/ Co – Pembimbing                            | : Hery, S. Kom, M.M.S.I. |
| <i>Student Name</i>      |                         | <i>Supervisor/Lecturer's Name/ Co-Supervising Lecturer's Name</i> |                          |
| NPM                      | : 01081230011           |                                                                   |                          |
| <i>Student ID Number</i> |                         |                                                                   |                          |
| Program Studi            | : Sistem Informasi      |                                                                   |                          |
| <i>Study Program</i>     |                         |                                                                   |                          |
| Fakultas                 | : FAIDAS                |                                                                   |                          |
| <i>Faculty</i>           |                         |                                                                   |                          |
| Semester                 | : Akselerasi 2026-2027  |                                                                   |                          |

| Minggu Ke-<br>Week Number | Hari<br>Day | Tanggal<br>Date | Materi yang didiskusikan<br>Discussed Material/Topics | Tanda tangan Dosen Pembimbing / Co-Pembimbing *)<br>Signature of Supervising Lecturer/Co-Supervising Lecturer *) | Catatan<br>Notes |
|---------------------------|-------------|-----------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------|
| 1                         | Senin       | 15 Juni 2026    | Bimbingan Format Proposal Tugas Akhir                 |                               |                  |
| 2                         | Kamis       | 9 Juli 2026     | Bimbingan Bab 1-2                                     |                               |                  |
| 3                         | Jumat       | 10 Juli 2026    | Bimbingan Bab 3 dan Finalisasi Bab 1-3                |                               |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |
|                           |             |                 |                                                       |                                                                                                                  |                  |

\*) Coret yang tidak perlu  
Cross out what is not necessary.

True Knowledge, Faith in Christ, Godly Character